智慧养老阿里云部署版本

1. 在阿里云中获取ECS服务器








选择实例
最近使用规格
全部规格
ECS 实例规格推荐助手
筛选
选择 vCPU
选择内存
模糊搜索规格名称
I/O 优化实例
查看更多规格参数
架构
X86 计算
Arm 计算
GPU/FPGA/ASIC
弹性裸金属服务器
高性能计算
全部分类
规格族
实例规格
vCPU
内存
可售可用区
架构-分类
处理器
本地存
选择实例: 当前建议在2CPU和4GB内存就足够
经济型 e 荐 ecs.e-c1m1.large 2 vCPU 2 GiB 67个可用区 X86 计算-共享型 intel
经济型 e 荐 ecs.e-c1m2.large 2 vCPU 4 GiB 66个可用区 X86 计算-共享型 intel
通用算力型 u1 荐 ecs.u1-c1m2.large 2 vCPU 4 GiB 61个可用区 X86 计算-通用算力型 intel
计算型 c7 荐 ecs.c7.xlarge 4 vCPU 8 GiB 42个可用区 X86 计算-计算型 intel
计算型 c7 荐 ecs.c7.large 2 vCPU 4 GiB 42个可用区 X86 计算-计算型 intel
经济型 e 荐 ecs.e-c1m2.xlarge 4 vCPU 8 GiB 65个可用区 X86 计算-共享型 intel
通用型 g7 荐 ecs.g7.large 2 vCPU 8 GiB 40个可用区 X86 计算-通用型 intel
计算型 c7 荐 ecs.c7.2xlarge 8 vCPU 16 GiB 42个可用区 X86 计算-计算型 intel
通用算力型 u1 ecs.u1-c1m1.large 2 vCPU 2 GiB 63个可用区 X86 计算-通用算力型 intel
通用算力型 u1 ecs.u1-c1m1.xlarge 4 vCPU 4 GiB 62个可用区 X86 计算-通用算力型 intel


实例使用时长
设定实例使用1小时
无确定使用时长
单台实例上限价格
使用自动出价
设置单台上限价
查看历史价格
实例中断模式
直接释放
节省停机
镜像
请输入关键词（例如 Ubuntu）快速查找镜像
镜像目录
最近使用镜像
公共镜像
自定义镜像
共享镜像
云市场镜像 荐
社区镜像
Alibaba Cloud Linux
Anolis OS
CentOS
Windows Server
SUSE Linux
Ubuntu
Debian
Fedora
OpenSUSE
Rocky Linux
CentOS Stream
AlmaLinux
Fedora CoreOS
FreeBSD
Red Hat
CentOS Stream 9 64位
免费安全加固 云服务器加载基础安全组件，提供网站漏洞检查、云产品安全配置检查、主机登录异常告警等安全功能，并可以通过云安全中心统一管理
镜像特性:
1. 该镜像支持使用ecs-user作为默认用户， 了解更多>
2. 该镜像rtc时钟默认使用utc标准， 了解更多>
扩展程序
暂无可用的扩展程序
选择不同实例规格时，系统会为您筛选可用的扩展程序。

存储

系统盘
如何选择云盘

默认 40GB空间 基本够用

类型	容量	数量	IOPS	性能	设置
ESSD Entry	40 GiB	1	2120	-	随实例释放 / 加密

云盘类型（**ESSD Entry**）在当前可用区已购买 **0** GiB，还可购买 **255880** GiB，提升配额

云盘性能：不同云盘性能不同，各云盘性能指标>
云盘容量：云盘创建总大小会受到配额限制 查看详情

数据盘
＋ 添加数据盘（0 / 16）

快照服务 荐
系统盘快照策略：请选择系统盘自动快照策略　创建自动快照策略

云盘数据存放在特定的可用区中，建议您使用快照服务为云盘创建备份，以抵御单机房故障的风险
快照服务特性：快照服务能定时对云盘进行备份。可应对病毒感染、数据误删等风险。 快照计费，包括快照存储容量以及复制费用（按量付费，每小时扣费）>
使用须知：
1. ⓘ此次应用的快照策略仅针对此次创建的云盘进行保护，后续新创建的云盘需要单独应用快照策略
2. ⓘ快照数据存放在 本地冗余 OSS，当快照所属可用区不可用时，会导致相关数据无法访问。如您的业务需要更高的可用性保障，建议您开启快照跨地域复制
ⓘ30天免费：您当前账号有快照免费30天权益 立即获取免费权益

▾ 弹性临时盘 | 文件存储 NAS（选填）

带宽和安全组

公网 IP
☑ 分配公网 IPv4 地址
系统会分配公网 IP，也可采用更加灵活的弹性公网 IP 方案，了解 如何配置并绑定弹性公网 IP 地址>

带宽计费模式
按固定带宽 适用于流量较大、稳定的场景
按使用流量 适用于流量小、波动大的场景
☐ 升级至CDT计费
开通云数据传输CDT，ⓘ赠送200GB/月公网流量 (其中20GB/月可用于中国内地地域)，超出部分采用 ⓘ阶梯计费，查看图文说明。价格详情 >

带宽峰值
1 | 2 | 3 | 5 | 10 | 50 | 100 Mbps　— 100 ＋ Mbps
阿里云免费提供最高 5Gbps 的恶意流量攻击防护。了解更多> | 提升防护能力>
ⓘ按使用流量计费模式下的出入带宽峰值都是带宽上限，不作为业务承诺指标。当出现资源争抢时，带宽峰值可能会受到限制。如果您的业务需要有带宽的保障，请使用按固定带宽计费模式

安全组
如何配置安全组
已有安全组 | 新建安全组

安全组名称
中州养老安全防火墙

安全组类型
普通安全组 | 企业级安全组　普通安全组和企业级安全组的对比差异>

开通IPv4端口/协议

☑ SSH (TCP：22)	☐ Telnet (TCP：23)	☑ HTTP (TCP：80)
☐ HTTPS (TCP：443)	☐ MS SQL (TCP：1433)	☐ Oracle (TCP：1521)
☐ MySQL (TCP：3306)	☐ RDP (TCP：3389)	☐ PostgreSQL (TCP：5432)
☐ Redis (TCP：6379)	☐ ICMP（IPv4）	

ⓘ选择 SSH (TCP：22) 将设置 0.0.0.0/0 作为源的规则，即允许所有 IP 地址通过22端口访问您的实例。我们建议您创建实例后将安全组规则设置为仅允许从已知的IP地址进行访问

▾ 弹性网卡 | IPv6（选填）

管理设置

登录凭证
密钥对 | 自定义密码 | 创建后设置
密钥对安全强度远高于常规自定义密码，可以避免暴力破解威胁，建议您使用密钥对创建实例

登录名
◉ root　○ ecs-user
root具有操作系统的最高权限，使用root作为登录名可能会导致安全风险，建议您使用ecs-user作为登录名。前往了解更多>

登录密码
Aa123456!

确认密码
Aa123456!

标签
如何设计标签
＋ 添加标签（0 / 20）
标签由区分大小写的键值对组成。您设置的标签将应用在本次创建的全部实例和云盘


配置概要
保存为启动模板
付费类型
抢占式实例
地域
华南 2（河源）
可用区
河源 可用区 B
网络类型
专有网络
专有网络
[默认]vpc-f8zhjtp2iujtli37a6zip
交换机
[默认]vsw-f8z1kkryfhvgofw3jm7o4
实例规格
经济型 e / ecs.e-c1m2.large (2 vCPU 4 GiB)
抢占式实例使用时长
无确定使用时长
单台实例规格上限价格
使用自动出价
镜像
CentOS Stream 9 64位(安全加固)
系统盘
ESSD Entry 40GiB
随实例释放
公网带宽
按使用流量 100Mbps
展开更多
购买实例数量
3
最小购买数量
3
当 ECS的 库存数量 < 最小购买数量，会创建失败；当 购买数量 >= 库存数量 >= 最小购买数量，会按照 库存数量 来创建
使用时限
设置自动释放服务时间
当前配置市场价格(总价)：￥0.142155/时
公网流量费用：￥0.80/GB
查看明细
点击确认下单即表示您已知悉并同意产品服务协议、服务等级协议以及本页面中您勾选过的产品专属条款（若有）
确认下单



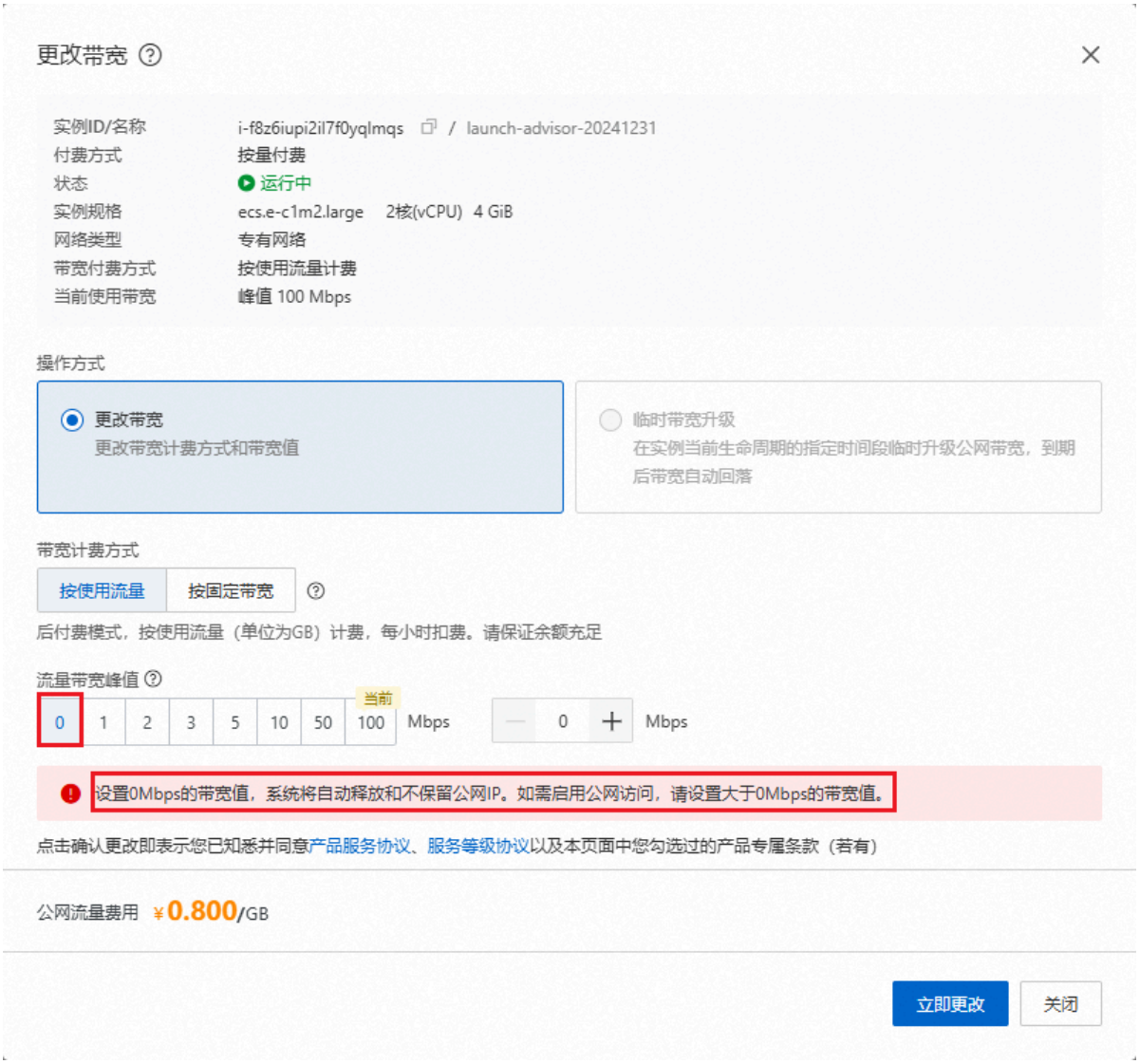


在这里需要关注几个点:

1- 阿里云服务器, 内网环境下,全部关闭了防火墙, 可以直接在内网环境下连接处理

2- 如果仅涉及内部数据交互, 无需要对外进行交互, 请注意,优先使用私有IP, 而非公有IP地址

3- 如果该服务器, 不涉及外部连接, 可以直接采用关闭公网方式,不允许外部网络连接



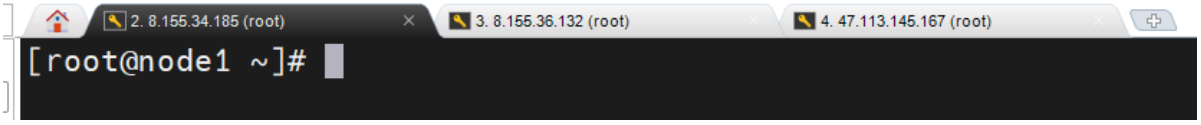
2. 基于MX连接三个服务器

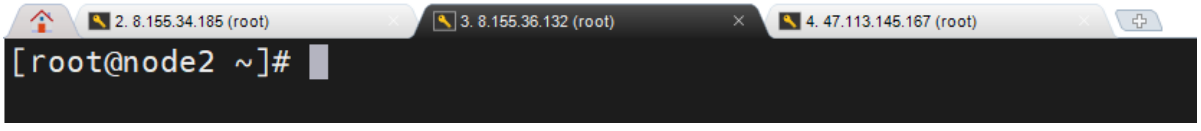
基于阿里云提供的各个节点的IP,完成连接操作

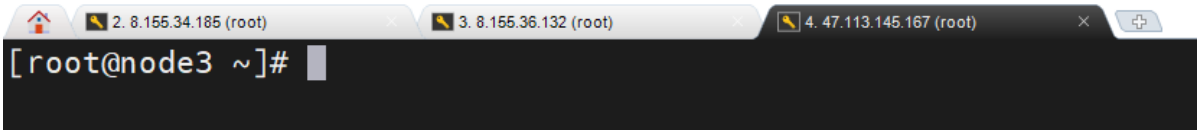


修改主机名字:

```
第一个节点: hostnamectl set-hostname node1.itcast.cn
第二个节点: hostnamectl set-hostname node2.itcast.cn
第三个节点: hostnamectl set-hostname node3.itcast.cn
```







设置/etc/hosts文件

设置SSH免密互通

防火墙默认已经关闭

SELinux默认也没有开启

静态IP是有阿里云分配也是固定的

时间同步无需要配置

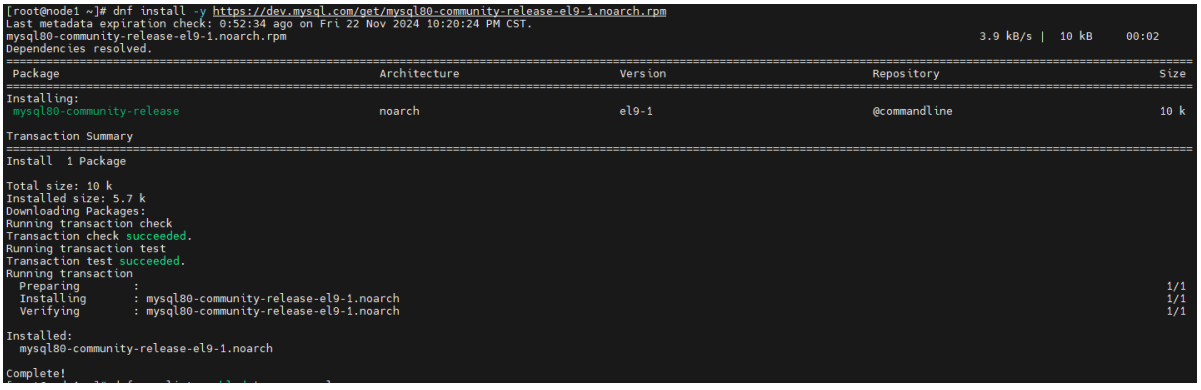
3- 配置数据库服务【node1】

3.1 安装MySQL

- 1- 安装MySQL官方dnf(yum)仓库:

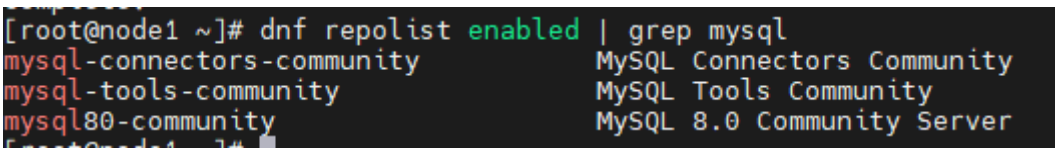
```
dnf install -y https://dev.mysql.com/get/mysql80-community-release-el9-1.noarch.rpm

# 原因： 默认的dnf仓库存在更新不及时，版本可能会老旧的情况
```



- 2- 查看是否启用MySQL8.0仓库

```
dnf repolist enabled | grep mysql
```

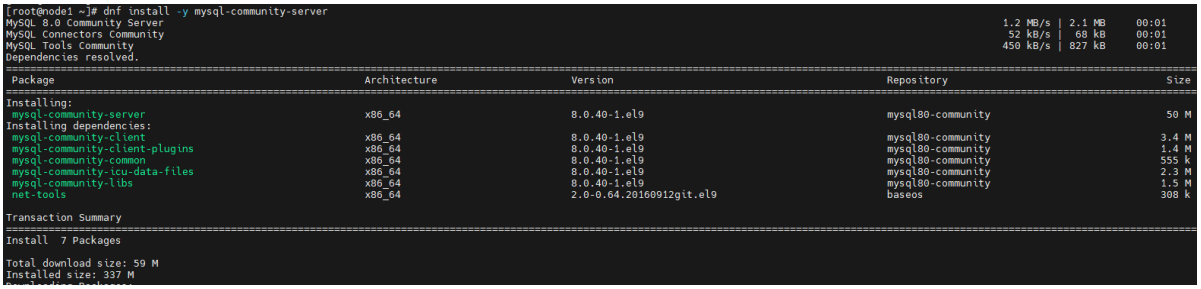


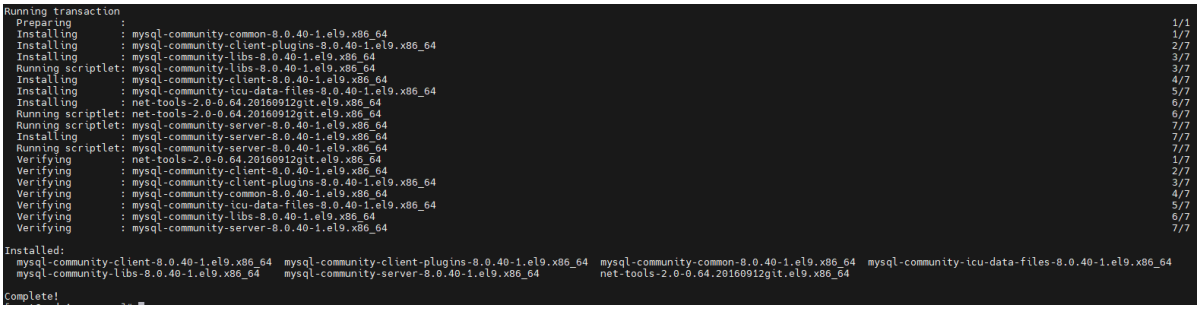
- 3- 安装MySQL8

```
dnf install -y mysql-community-server --nogpgcheck
```

--nogpgcheck ： 跳过 GPG 检查,当安装链接是可信的，建议直接跳过，避免由于网络等跟其他原因导致无法安装

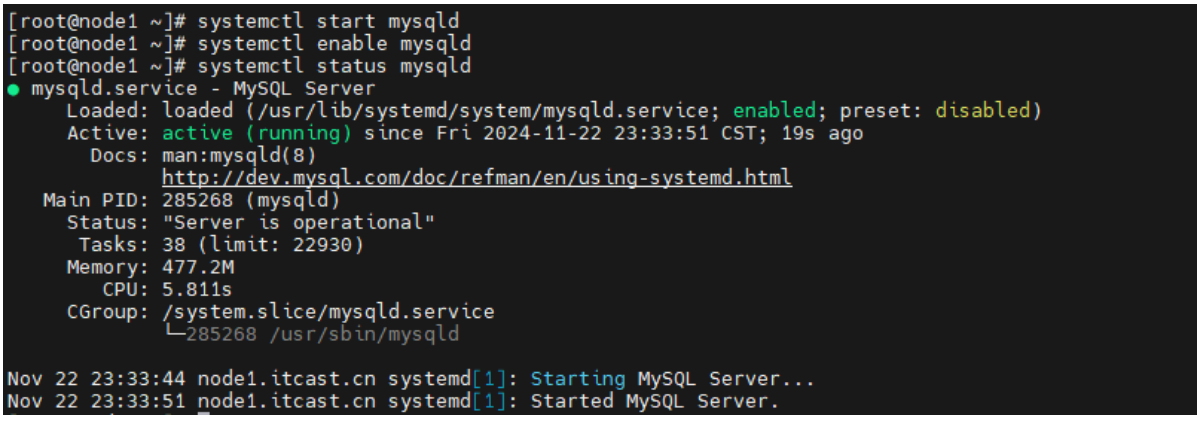
GPG（GNU Privacy Guard）是一种开源的加密工具，用于保障数据的安全性和真实性，主要用于加密、解密和数字签名验证。它的核心作用是通过公钥和私钥的机制提供数据完整性和防篡改验证。





- 4- 启动MySQL相关服务

```
systemctl start mysqld
systemctl enable mysqld
systemctl status mysqld
```



- 5- 获取MySQL初始root密码

  安装 MySQL 后，会生成一个随机的 root 密码, 可以通过MySQL的启动日志来查看

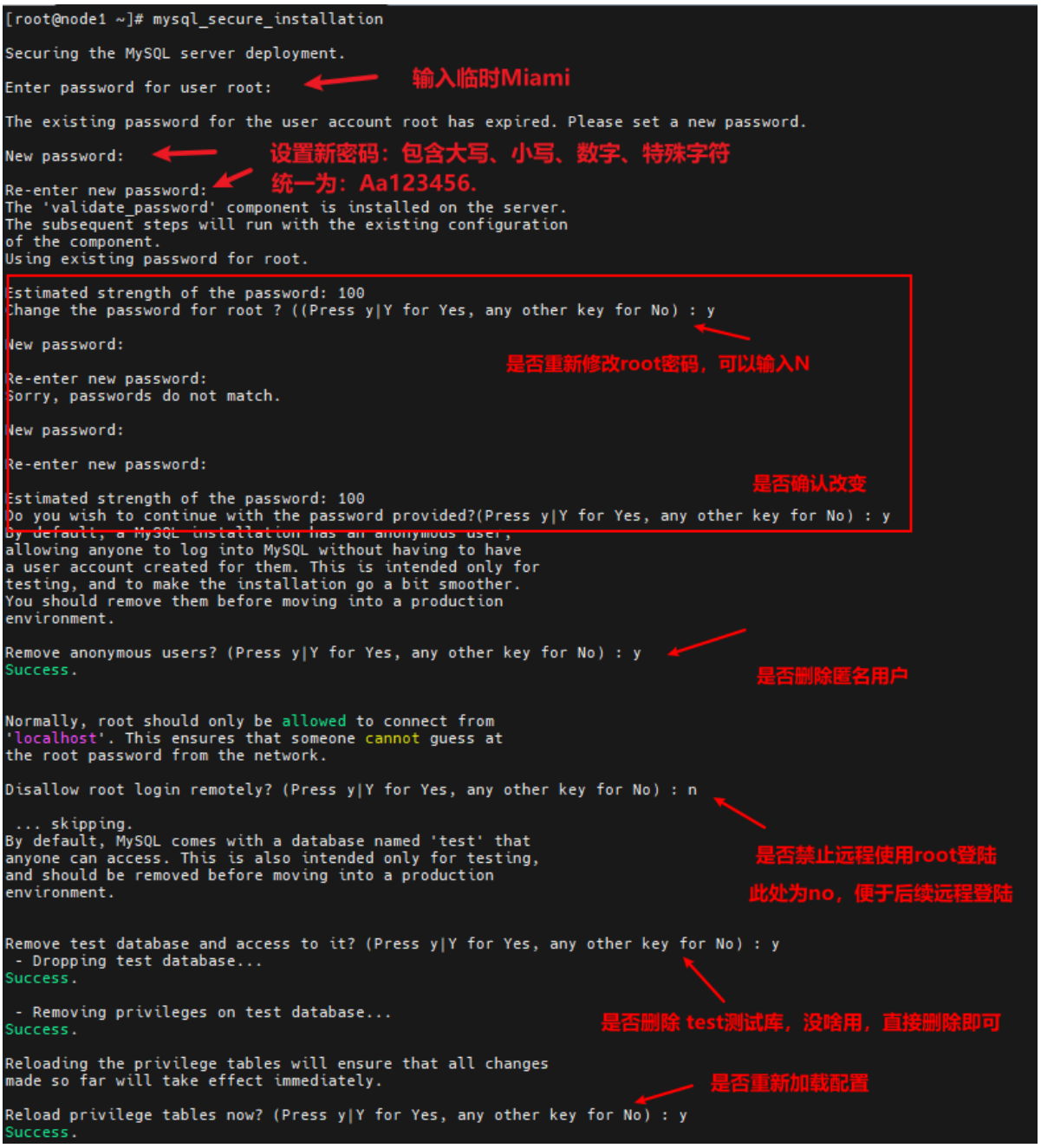
```
日志放置位置： /var/log/mysqld.log

grep password /var/log/mysqld.log
```



- 6- 登陆MySQL，重新设置密码

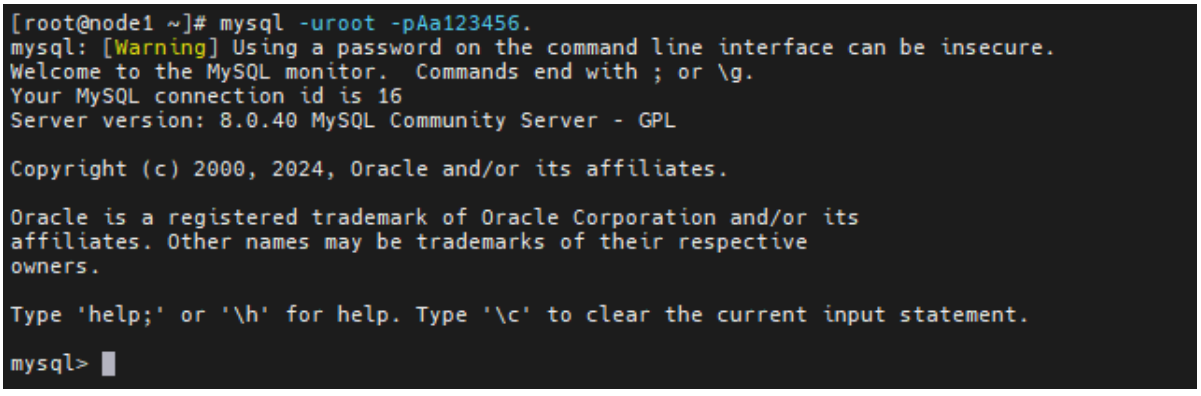
```
mysql_secure_installation
```



- 7- 测试登陆

```
mysql -uroot -p[密码]    可以省略密码 直接回车，然后输入密码

注意：此密码就是最新设置的 Aa123456.
```



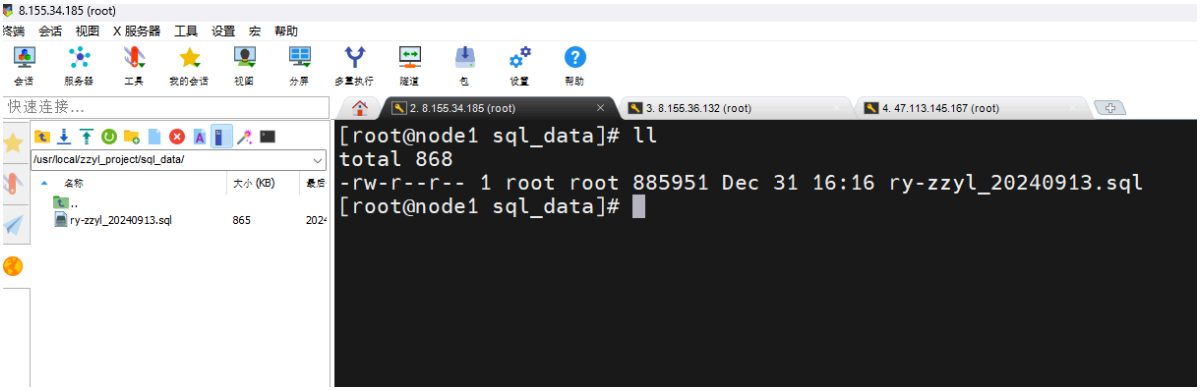

```
quit 或 crtl +d 退出
```

3.2 导入基础数据

- 1- 上传脚本到Linux环境中

```
mkdir -p /usr/local/zzyl_project/sql_data

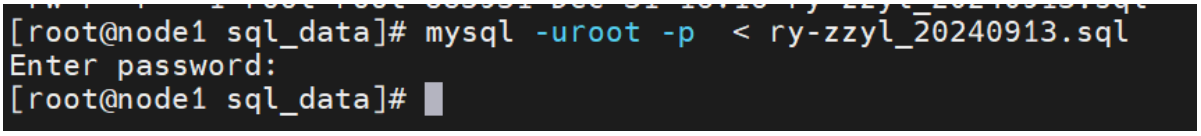
将SQL脚本上传到Linux的此目录下
```



- 2- 执行SQL脚本,导入数据

```
cd /usr/local/zzyl_project/sql_data
mysql -uroot -p  < ry-zzyl_20240913.sql

输入密码后，就会自动执行导入操作
```



验证:

```
mysql -uroot -p  -- 登录MySQL

show databases; -- 查看所有的库

use ry-zzyl;  -- 使用ry-zzyl

show tables; -- 使用所有表
```

```
[root@node1 sql_data]# mysql -uroot -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.0.40 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

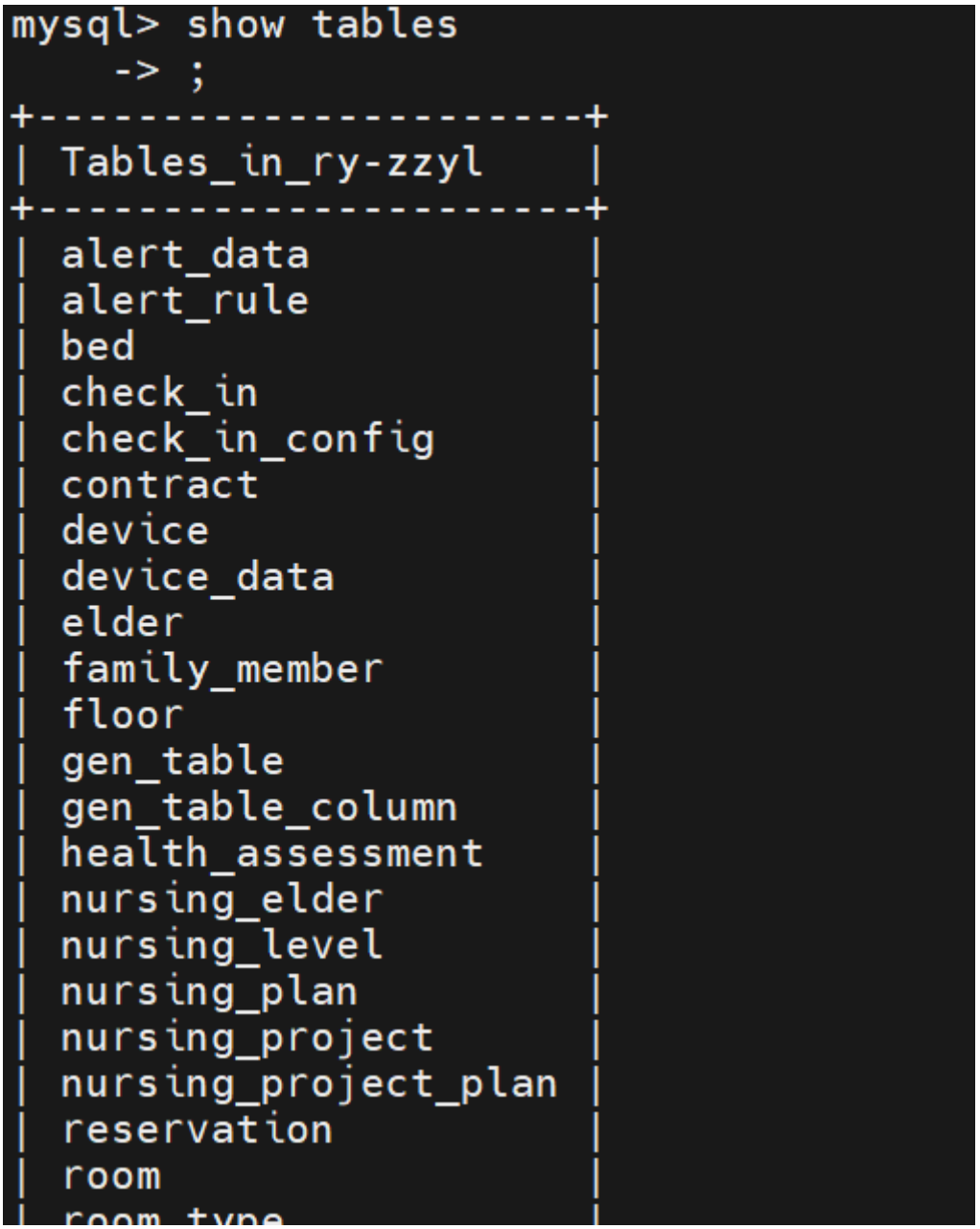
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| ry-zzyl            |
| sys                |
+--------------------+
5 rows in set (0.01 sec)

mysql> use ry-zzyl
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
```



4- 配置Redis服务【node1】

部署操作:

- 步骤一: 安装Redis服务

```
# 更新dnf相关软件包 【可选】  时间较长 预计20分钟
dnf update -y
# 安装redis操作
dnf install redis -y
```

```
[root@node1 ~]# dnf install redis -y
上次元数据过期检查：1:28:48 前，执行于 2024年12月27日 星期五 19时15分12秒。
依赖关系解决。
================================================================================
 软件包                架构          版本                    仓库          大小
================================================================================
安装:
 redis                 x86_64        6.2.16-1.el9            appstream     1.3 M

事务概要
================================================================================
安装  1 软件包

总下载：1.3 M
安装大小：4.7 M
下载软件包：
redis-6.2.16-1.el9.x86_64.rpm                   671 kB/s | 1.3 MB     00:01
--------------------------------------------------------------------------------
总计                                            366 kB/s | 1.3 MB     00:03
运行事务检查
事务检查成功。
运行事务测试
事务测试成功。
运行事务
  准备中  :                                                        1/1
  运行脚本: redis-6.2.16-1.el9.x86_64                              1/1
  安装    : redis-6.2.16-1.el9.x86_64                              1/1
  运行脚本: redis-6.2.16-1.el9.x86_64                              1/1
  验证    : redis-6.2.16-1.el9.x86_64                              1/1

已安装:
  redis-6.2.16-1.el9.x86_64

完毕！
[root@node1 ~]#
```

- 步骤二： 修改Redis相关配置

```
vim  /etc/redis/redis.conf
# 83行附件， 修改为  * -::*  任意的服务都可以连接redis服务
bind * -::*

#908行附近： 打开requirepass，设置其密码为123456 【可选】
requirepass 123456
```

- 步骤三： 启动redis进程

```
systemctl start redis
systemctl enable redis

查看状态：
systemctl status redis
```

```
[root@node1 ~]# systemctl status redis
● redis.service - Redis persistent key-value database
     Loaded: loaded (/usr/lib/systemd/system/redis.service; enabled; preset: disabled)
    Drop-In: /etc/systemd/system/redis.service.d
             └─limit.conf
     Active: active (running) since Fri 2024-12-27 20:52:58 CST; 41s ago
   Main PID: 124552 (redis-server)
     Status: "Ready to accept connections"
      Tasks: 5 (limit: 22929)
     Memory: 7.7M
        CPU: 65ms
     CGroup: /system.slice/redis.service
             └─124552 "/usr/bin/redis-server *:6379"

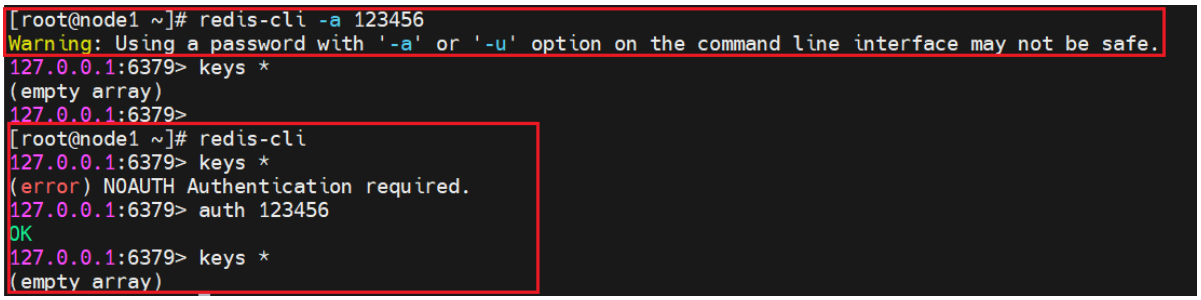
12月 27 20:52:58 node1.itcast.cn systemd[1]: Starting Redis persistent key-value database...
12月 27 20:52:58 node1.itcast.cn systemd[1]: Started Redis persistent key-value database.
```

- 步骤四： 测试redis服务

```
redis-cli 【-a 密码】

或
redis-cli  命令 进入客户端， 然后输入 auth 123456

退出客户端： quit 或 ctrl +d
```



5- 部署阿里云OSS

阿里云的 **OSS**（Object Storage Service）是一个 **云端对象存储服务**，简单来说，它就像是一个 **网上的硬盘**，用来存储和管理各种文件（比如图片、视频、文档等）。你可以把它理解为一个 **可以随时访问、随时上传文件的网盘**，不过它比普通网盘更强大，专门为大规模的数据存储和管理设计。

特点：

- **像硬盘，但在云上**：你把文件存到阿里云的OSS上，就像把文件放进云端的一个“网盘”中，而这个网盘可以存很多很多数据。
- **可以随时随地访问**：无论你身处哪里，只要有网络，就可以随时上传、下载、管理这些文件。
- **适合大规模数据存储**：不仅可以存普通文件，还能处理海量数据，比如网站的图片、视频，甚至是备份文件等。

- 步骤一：在阿里云中开通OSS服务



购买中： 仅需要选择开通服务即可

- 注意： 这里不用写立即购买， 选择立即开通即可， 一旦开通了， 就没有这个按钮了



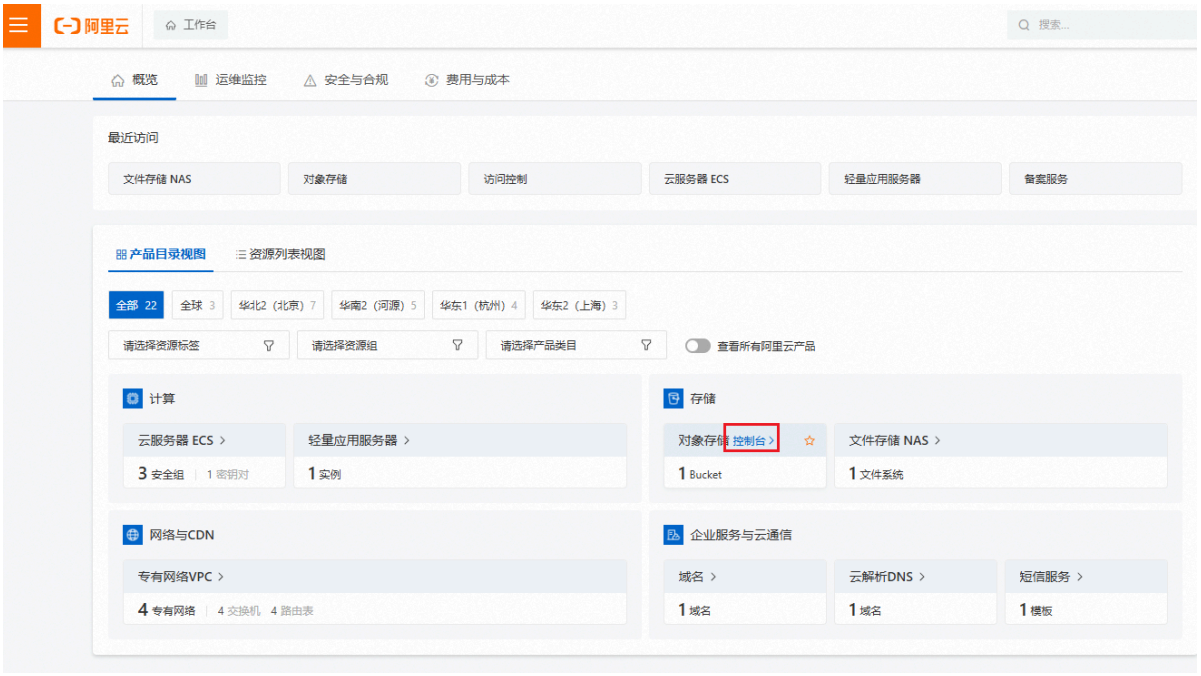
如果是第一次使用， 可以直接选择免费试用

点击立即按钮，填写配置信息，单击立即使用即可：

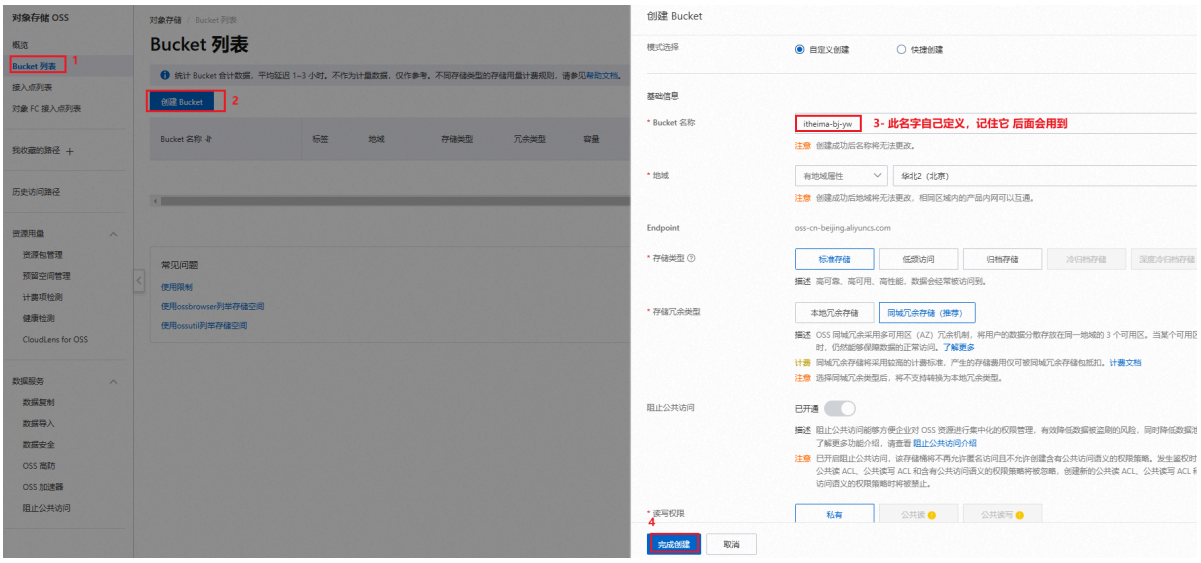


- 步骤二： 进入到对象存储管理控制台





- 步骤三： 创建存储backet桶（容器）

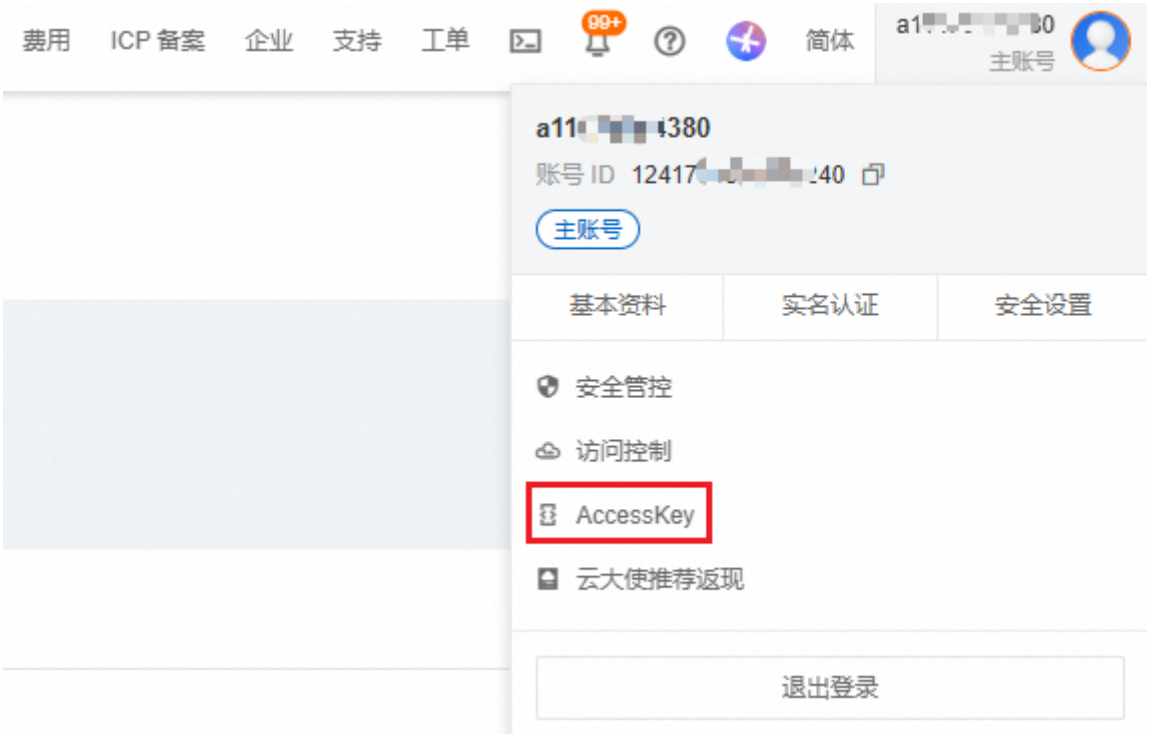


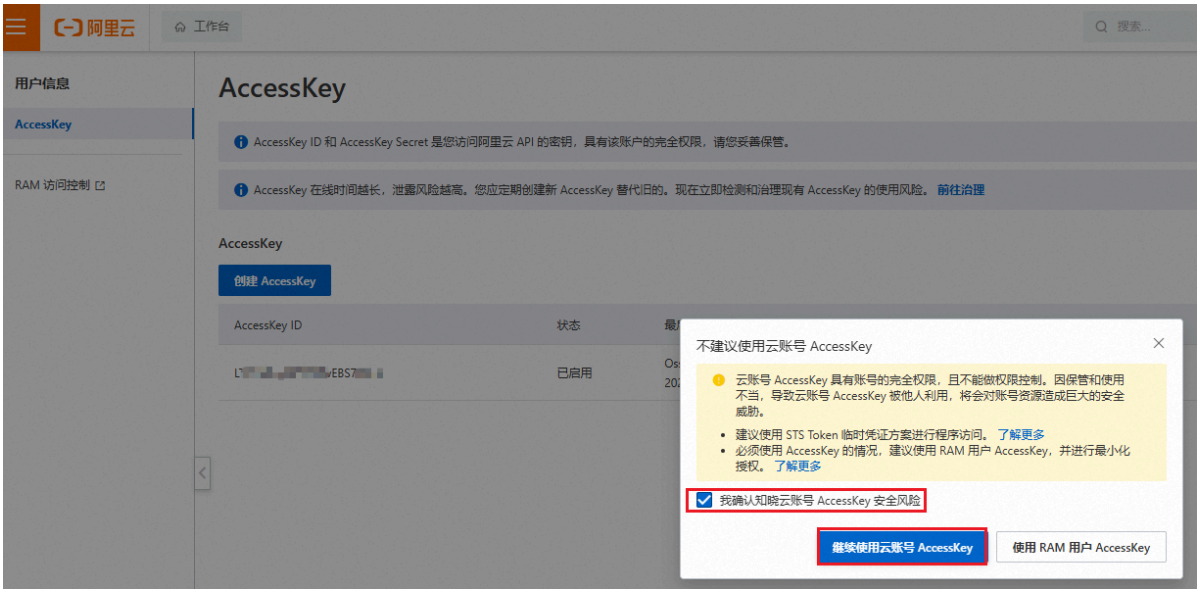
- 步骤四： 进入bucket， 设置为公用， 方便访问

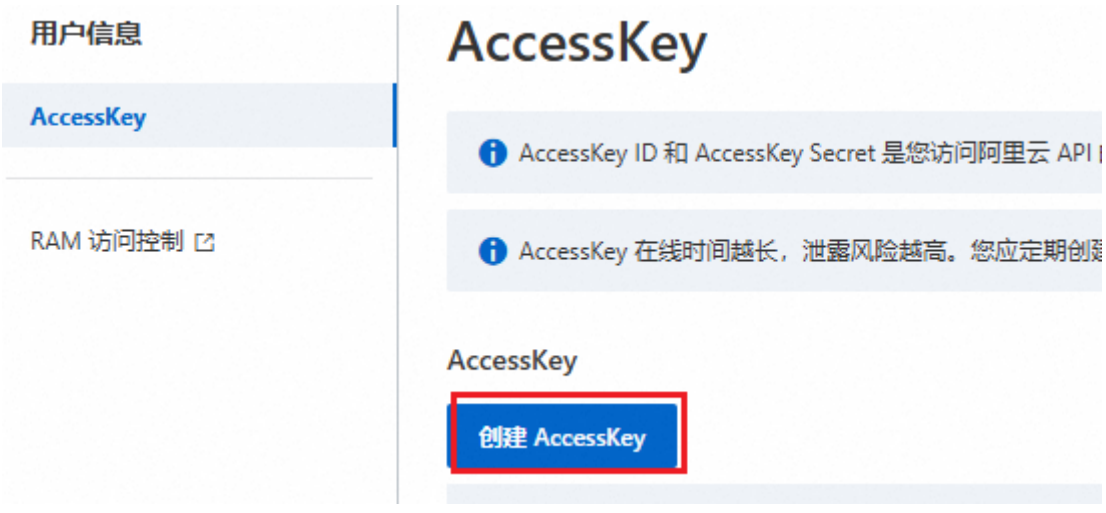




- 生成访问accessKey，用于授权访问OSS服务


费用
ICP 备案
企业
支持
工单
99+
简体
主账号
主账号
基本资料
实名认证
安全设置
安全管控
访问控制
AccessKey
云大使推荐返现
退出登录

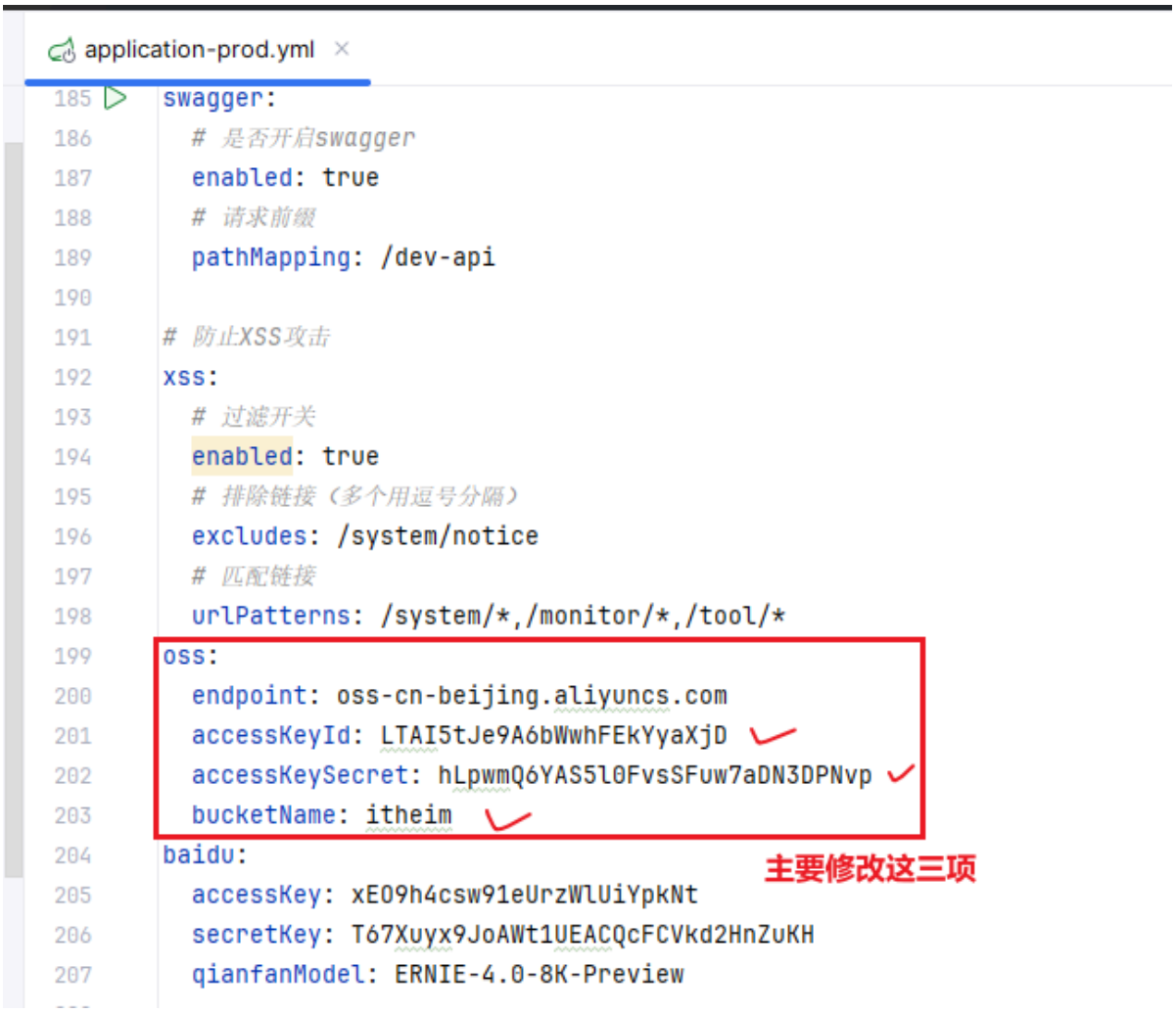

阿里云
工作台
用户信息
AccessKey
RAM 访问控制
AccessKey
AccessKey ID 和 AccessKey Secret 是您访问阿里云 API 的密钥，具有该账户的完全权限，请您妥善保管。
AccessKey 在线时间越长，泄露风险越高。您应定期创建新 AccessKey 替代旧的。现在立即检测和治理现有 AccessKey 的使用风险。 前往治理
AccessKey
创建 AccessKey
AccessKey ID
状态
已启用
不建议使用云账号 AccessKey
云账号 AccessKey 具有账号的完全权限，且不能做权限控制。因保管和使用不当，导致云账号 AccessKey 被他人利用，将会对账号资源造成巨大的安全威胁。
建议使用 STS Token 临时凭证方案进行程序访问。 了解更多
必须使用 AccessKey 的情况，建议使用 RAM 用户 AccessKey，并进行最小化授权。 了解更多
我确认知晓云账号 AccessKey 安全风险
继续使用云账号 AccessKey
使用 RAM 用户 AccessKey


用户信息
AccessKey
RAM 访问控制
AccessKey
AccessKey ID 和 AccessKey Secret 是您访问阿里云 API
AccessKey 在线时间越长，泄露风险越高。您应定期创
AccessKey
创建 AccessKey



- 步骤五： 修改项目中关于OSS服务的配置信息

```
accessKeyId: 授权ID
accessKeySecret: 授权Secret
bucketName: 刚刚在oss创建的bucket
```



6- 申请百度千帆大模型授权【可选】

项目中, 主要是基于百度千帆大模型来分析用户的体检报告, 故需要注册百度千帆大模型APK, 从而对接百度千帆大模型. 生产环境中, 此位置一般选择为公司生产环境的百度千帆账号

- 注册地址：https://qianfan.cloud.baidu.com/



实名认证:

有了账号之后，我们需要**个人实名认证，**不然大模型调用不了，其中实名认证大家需要在手机端进行操作

1. 在手机上下载一个百度智能云app



2. 使用刚刚注册的账号进行登录，找到**我的**，**个人中心**，可以进行实名认证操作



注册后, 赠送20元优惠卷, 有效期1个月

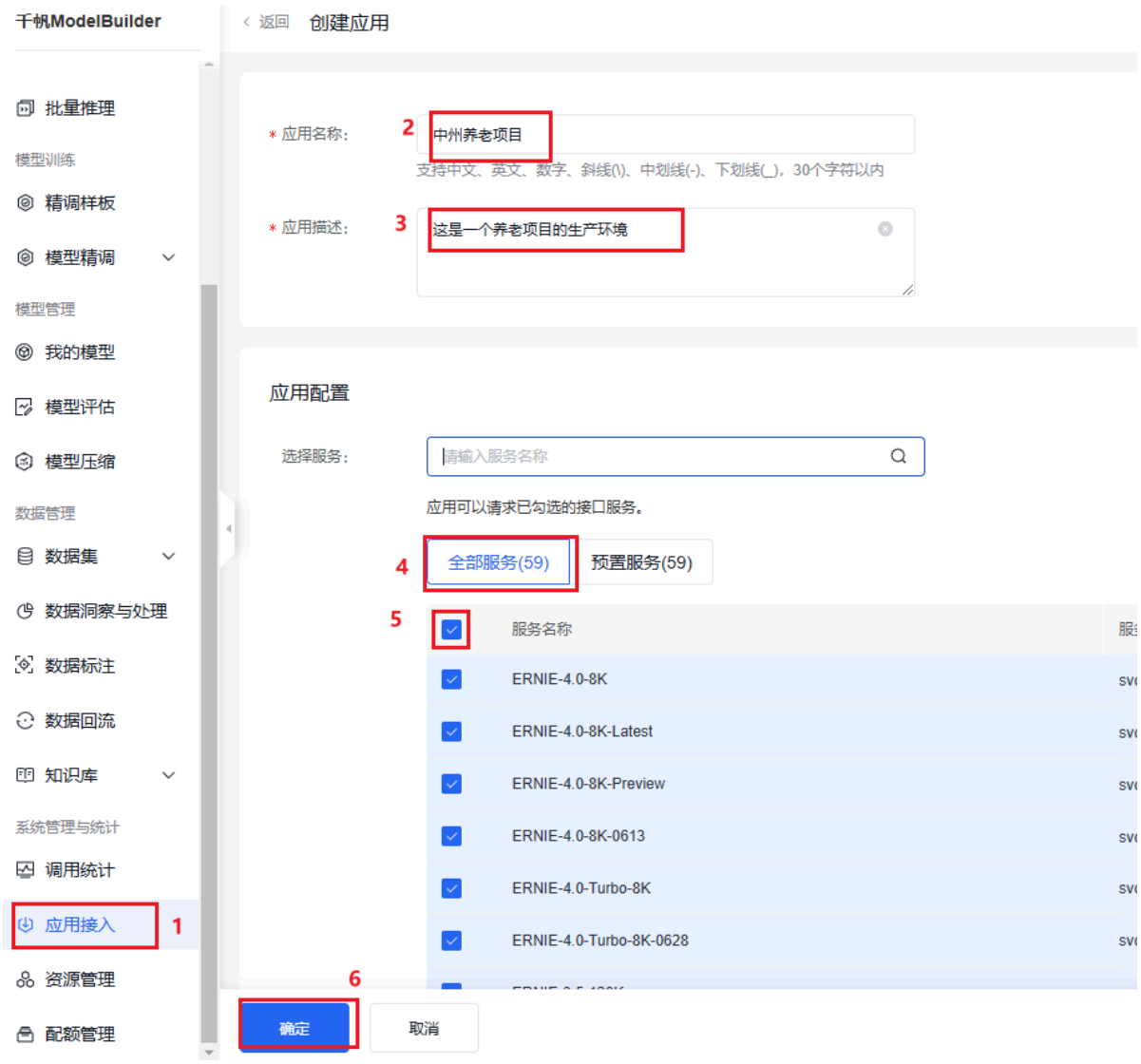
创建应用

实名认证成功之后，我们继续在PC端来访问千帆大模型，地址：https://qianfan.cloud.baidu.com/

进入到管理平台



进入管理平台后，找到**应用接入**，我们需要创建新的应用，只有创建了应用，后面才能让大模型来绑定应用使用



应用创建之后，可以查看自己的应用信息，其中就包含了一些秘钥信息，不要随意泄漏，如下图



设置应用到项目中:

```
accessKey: API KEY
secretKey: 对应的secret key
qianfanModel: ERNIE-4.0-8K-Preview(模型选择)
```

application-prod.yml

```
swagger:
  # 是否开启swagger
  enabled: true
  # 请求前缀
  pathMapping: /dev-api

# 防止XSS攻击
xss:
  # 过滤开关
  enabled: true
  # 排除链接（多个用逗号分隔）
  excludes: /system/notice
  # 匹配链接
  urlPatterns: /system/*,/monitor/*,/tool/*
oss:
  endpoint: oss-cn-bei[illegible]uncs.com
  accessKeyId: LTAI[illegible]ydEmyj2VY
  accessKeySecret: bQ2q[illegible]ygVfDaTEYUdwA3gUkEJiKb
  bucketName: ithei[illegible]oj-yw
baidu:
  accessKey: xEO9h[illegible]zWlUiYpkNt
  secretKey: T67Xuyx[illegible]UEACQcFCVkd2HnZuKH
  qianfanModel: ERNIE-4.0-8K-Preview
```

7- 后台管理系统打包部署

7.1 在node2中部署JDK11

- 1- 在node2中创建放置jdk11的安装包目录

```
mkdir -p /export/software
```

- 2- 将资料中提供的jdk11的安装包上传到此目录下 或联网下载即可

```
wget https://download.java.net/openjdk/jdk11.0.0.2/ri/openjdk-11.0.0.2_linux-x64.tar.gz
```

```
[root@node2 ~]# cd /export/software/
[root@node2 software]# ll
总用量 182644
-rw-r--r-- 1 root root 187027091 12月 29 11:36 openjdk-11.0.0.2_linux-x64.tar.gz
[root@node2 software]#
```

- 3- 对jdk11安装包进行解压： 解压到 /usr/local

```
tar -xzf openjdk-11.0.0.2_linux-x64.tar.gz -C /usr/local/

cd /usr/local

输入 ll 查看
```

```
[root@node2 software]# tar -xzf openjdk-11.0.0.2_linux-x64.tar.gz -C /usr/local/
[root@node2 software]# cd /usr/local/
[root@node2 local]# ll
总用量 0
drwxr-xr-x. 2 root root  6  6月 25  2024 bin
drwxr-xr-x. 2 root root  6  6月 25  2024 etc
drwxr-xr-x. 2 root root  6  6月 25  2024 games
drwxr-xr-x. 2 root root  6  6月 25  2024 include
drwxr-xr-x  8 root root 96 12月 29 11:37 jdk-11.0.0.2
drwxr-xr-x. 2 root root  6  6月 25  2024 lib
drwxr-xr-x. 3 root root 17 11月 29 15:12 lib64
drwxr-xr-x. 2 root root  6  6月 25  2024 libexec
drwxr-xr-x. 2 root root  6  6月 25  2024 sbin
drwxr-xr-x. 5 root root 49 11月 29 15:12 share
drwxr-xr-x. 2 root root  6  6月 25  2024 src
drwxr-xr-x  2 root root 28 12月 28 23:42 zzyl_project
[root@node2 local]#
```

- 4- 配置JDK的环境变量

```
vim /etc/profile

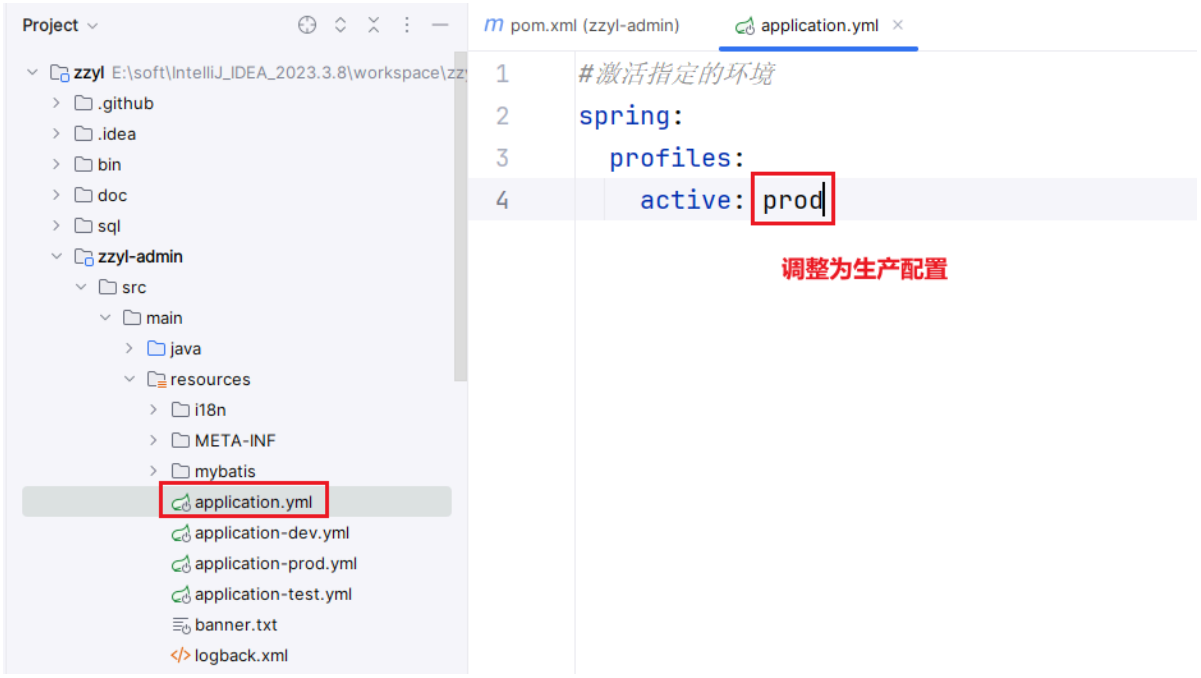
在文件的尾部添加：
# JAVA_HOME
export JAVA_HOME=/usr/local/jdk-11.0.0.2
export PATH=$JAVA_HOME/bin:$PATH
export CLASSPATH=$JAVA_HOME/lib:$CLASSPATH


保存退出后，执行加载操作：
source /etc/profile
```

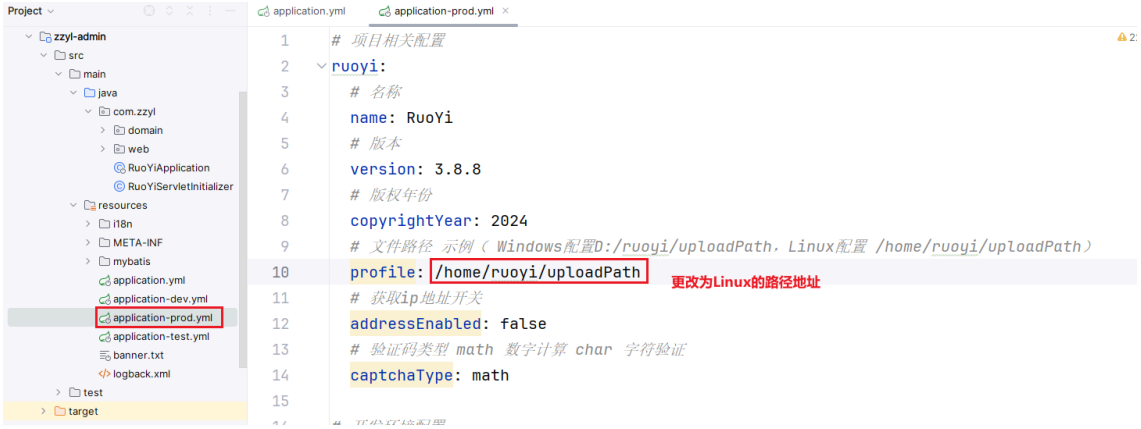
```
[root@node2 local]# source /etc/profile
[root@node2 local]# java -version
openjdk version "11.0.0.2" 2024-07-02
OpenJDK Runtime Environment 18.9 (build 11.0.0.2+2-2)
OpenJDK 64-Bit Server VM 18.9 (build 11.0.0.2+2-2, mixed mode)
```

9.2 生产环境配置操作

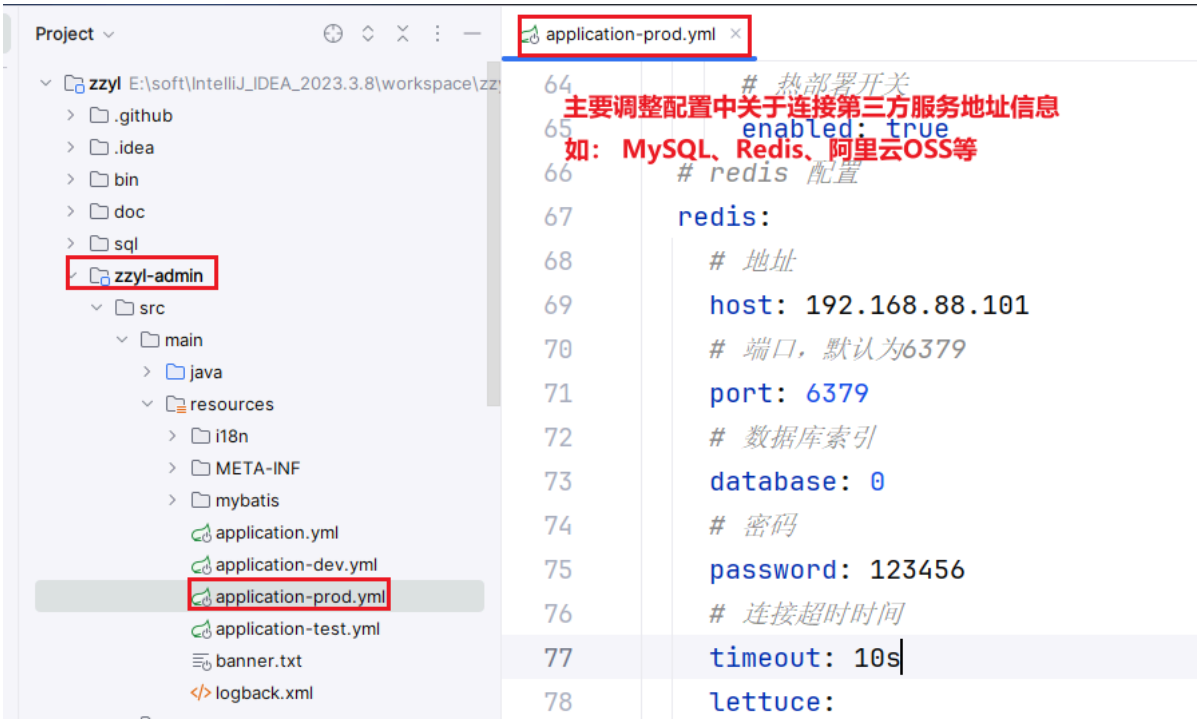
- 1- 调整运行模式为生产环境配置



- 2- 修改生产环境配置连接信息， 确保可以正常连接到生产环境服务器



注意: 在配置Mysql Redis地址都需要配置为对应服务器的私有IP地址即可, 无需要配置公网IP

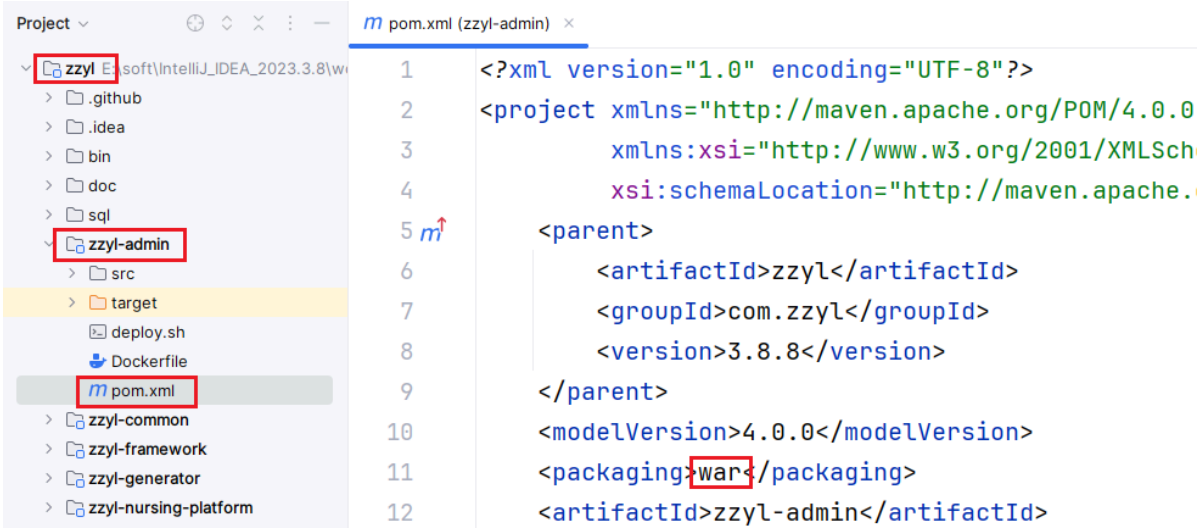


application.yml　application-prod.yml

```yaml
  # 过滤开关
  enabled: true
  # 排除链接（多个用逗号分隔）
  excludes: /system/notice
  # 匹配链接
  urlPatterns: /system/*,/monitor/*,/tool/*
oss:
  endpoint: oss-cn-beijing.aliyuncs.com
  accessKeyId: LTAI5tG4QB46R4yvBtt4FwRm
  accessKeySecret: 3PaP3bTIUoeOV71FlBEg6n3j86Bo4h
  bucketName: itheima-bj-0603
baidu:
  accessKey: Lhk2UbGRV3Q5chV7iFxx1ljA
  secretKey: rEne2BFlnG8v2oHvupWt4OvO5VUuSc8N
  qianfanModel: ERNIE-4.0-8K-Preview
```

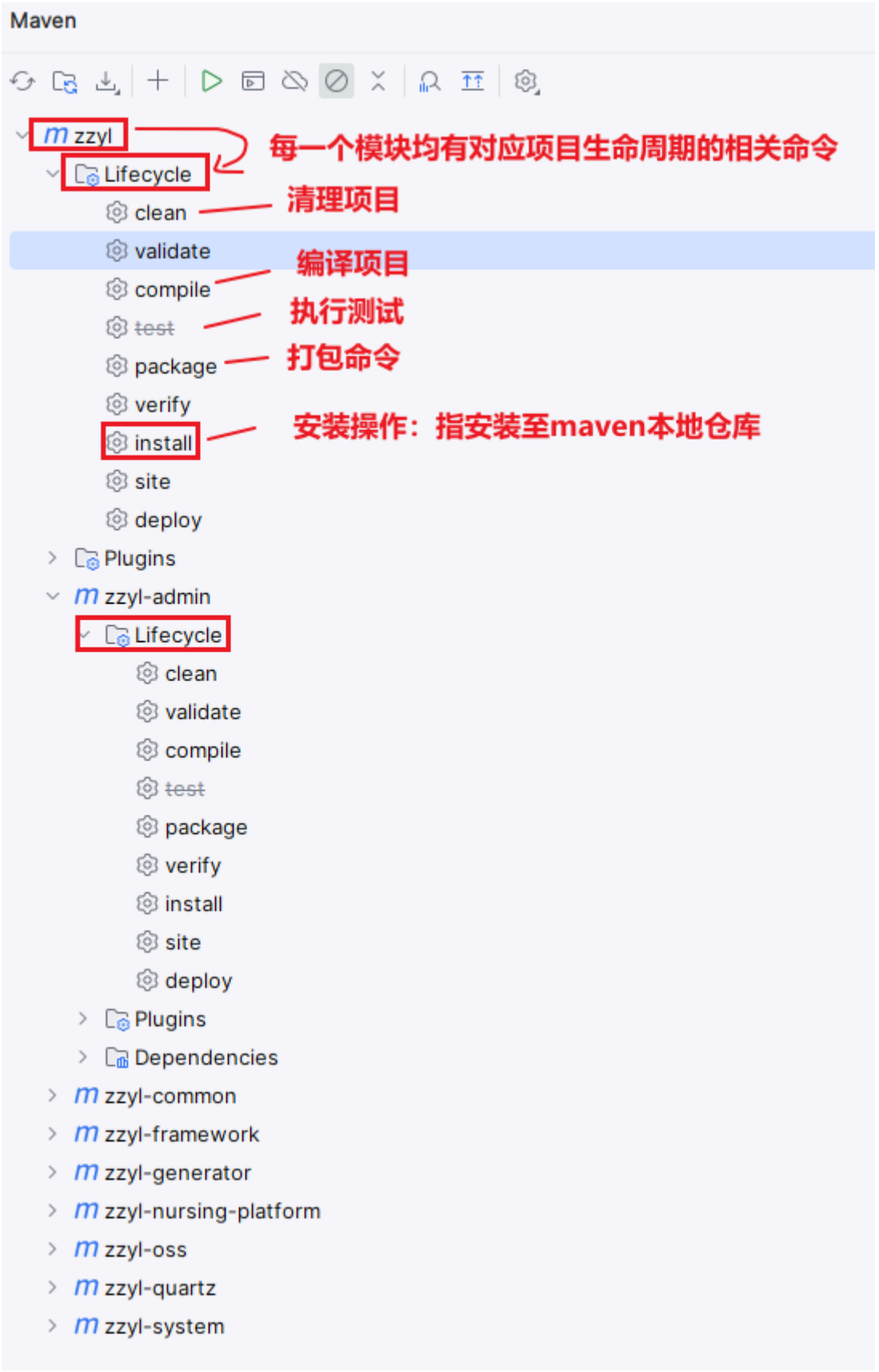
9.4 方式二：基于tomcat运行

如何实现呢?

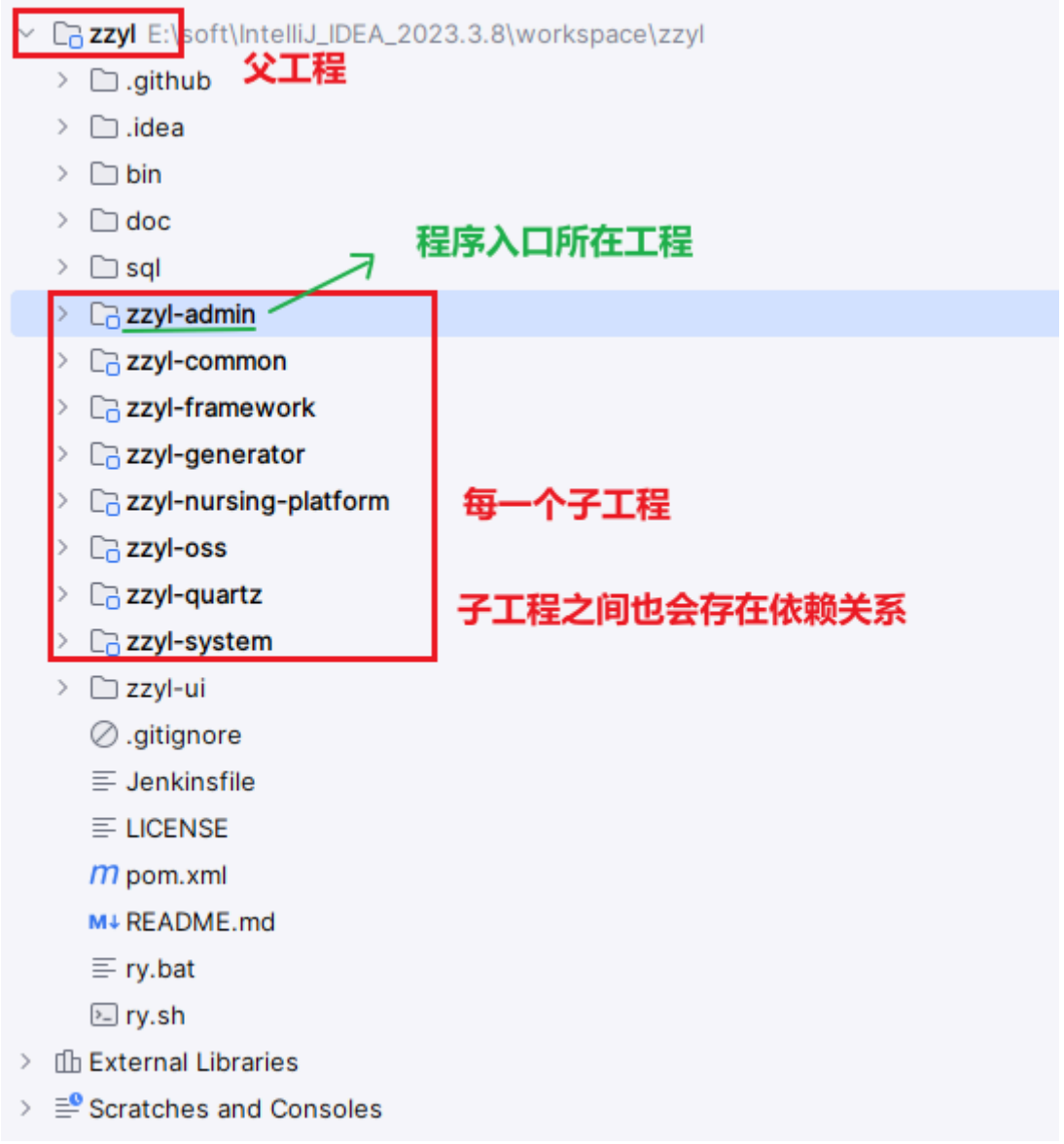


执行打包操作:

对于Maven项目来说， 本身提供了关于项目管理的相关命令，这里就包括打包命令

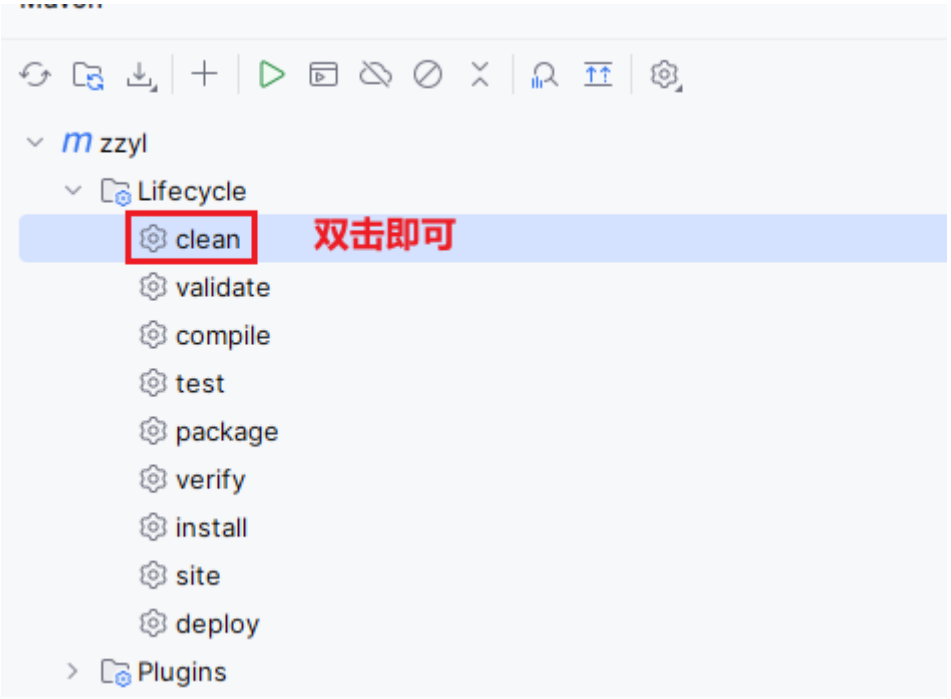

Maven
zzyl
每一个模块均有对应项目生命周期的相关命令
Lifecycle
clean
清理项目
validate
compile
编译项目
test
执行测试
package
打包命令
verify
install
安装操作：指安装至maven本地仓库
site
deploy
Plugins
zzyl-admin
Lifecycle
clean
validate
compile
test
package
verify
install
site
deploy
Plugins
Dependencies
zzyl-common
zzyl-framework
zzyl-generator
zzyl-nursing-platform
zzyl-oss
zzyl-quartz
zzyl-system

本项目结构：



对项目进行打包： 由于项目之间存在依赖关系， 我们无法之间仅在程序入口工程进行打包， 此时可以选择之间针对父工程打包

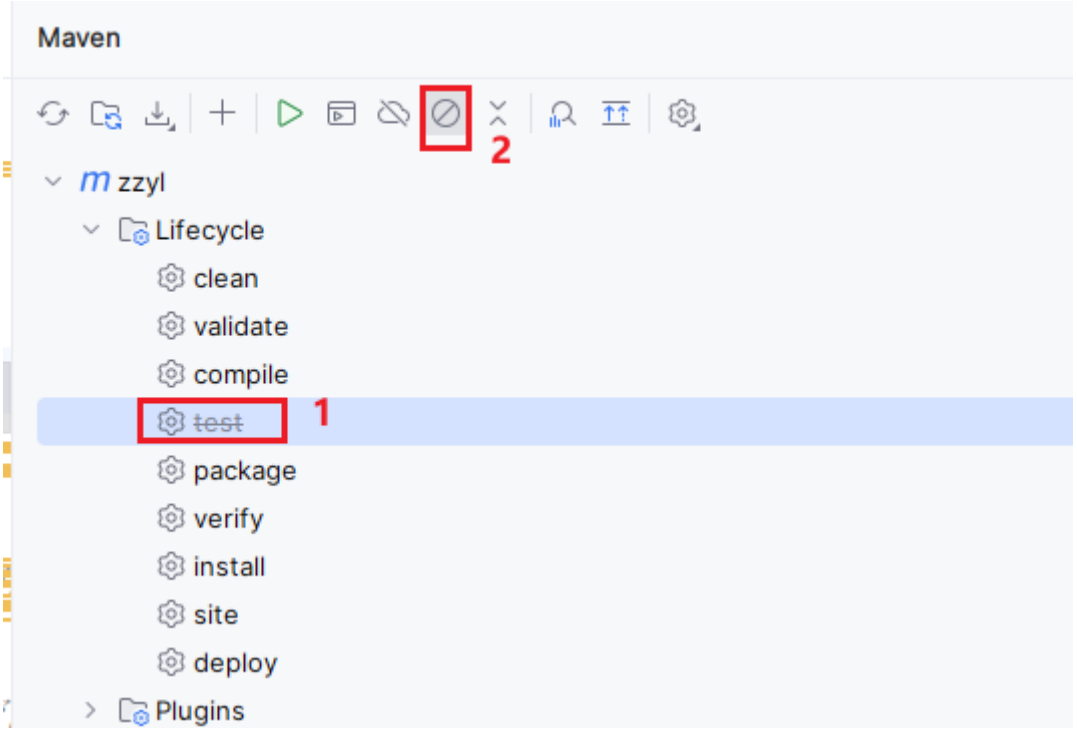
- 第一步： 先执行清理操作， 确保项目无杂质历史信息



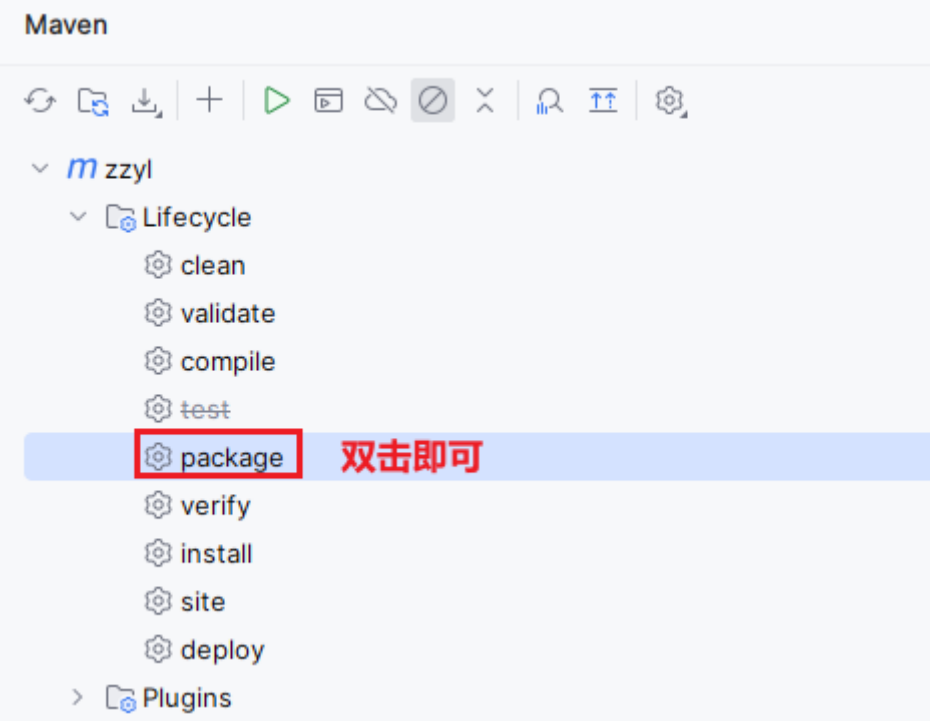
执行日志可以看到:

```
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ zzyl-admin ---
[INFO] Deleting E:\soft\IntelliJ_IDEA_2023.3.8\workspace\zzyl\zzyl-admin\target
[INFO] ------------------------------------------------------------------------
[INFO] Reactor Summary for zzyl 3.8.8:
[INFO]
[INFO] zzyl ........................................... SUCCESS [  0.139 s]
[INFO] zzyl-common .................................... SUCCESS [  0.080 s]
[INFO] zzyl-system .................................... SUCCESS [  0.035 s]
[INFO] zzyl-framework ................................. SUCCESS [  0.042 s]
[INFO] zzyl-quartz .................................... SUCCESS [  0.019 s]
[INFO] zzyl-generator ................................. SUCCESS [  0.039 s]
[INFO] zzyl-oss ....................................... SUCCESS [  0.017 s]
[INFO] zzyl-nursing-platform .......................... SUCCESS [  0.045 s]
[INFO] zzyl-admin ..................................... SUCCESS [  0.119 s]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

- 第二步： 去除test命令
  - 默认情况下, 在执行打包之前, 会先运行编辑和测试操作, 建议在正式打包前,将测试命令从流程中删除

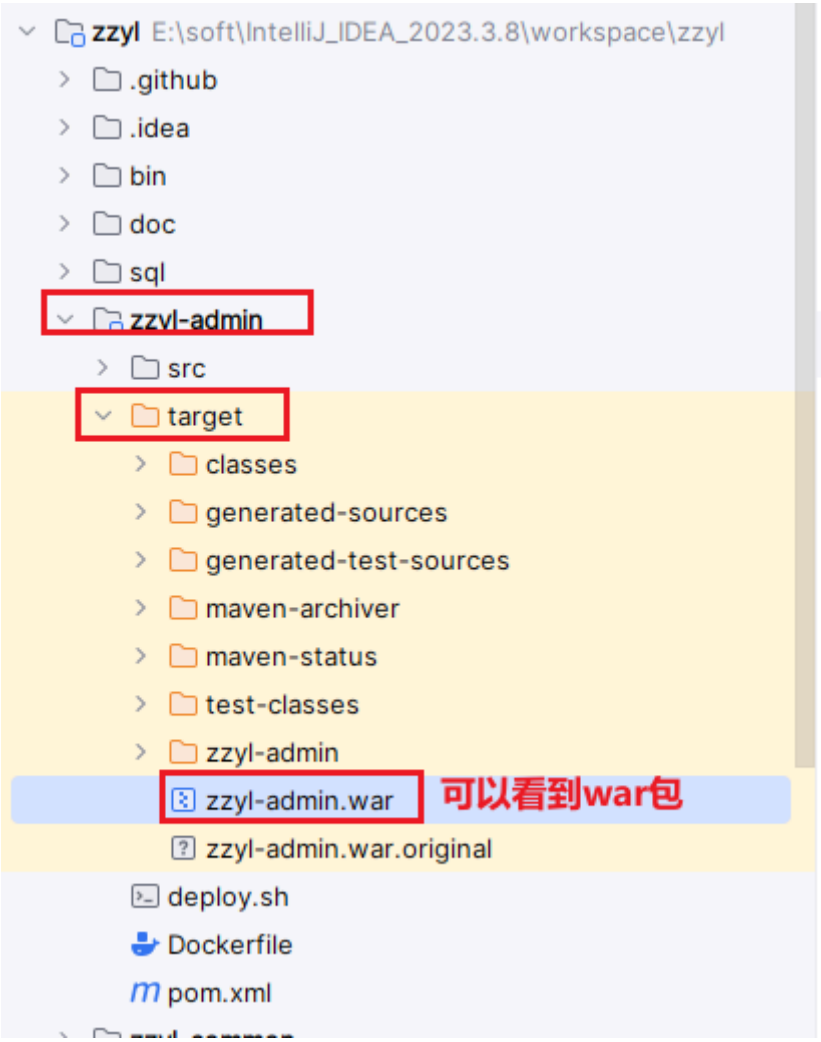


- 第三步: 执行打包操作

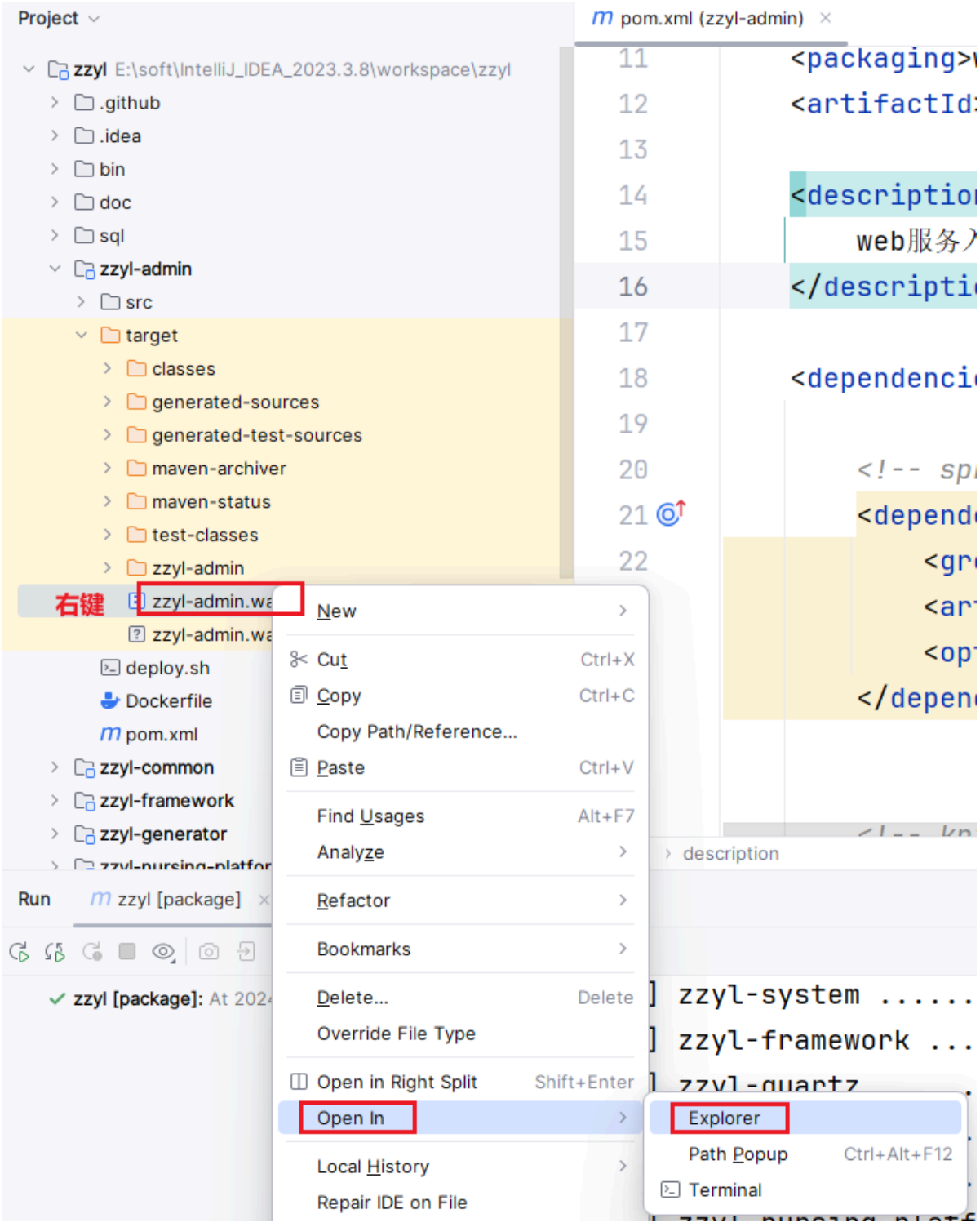


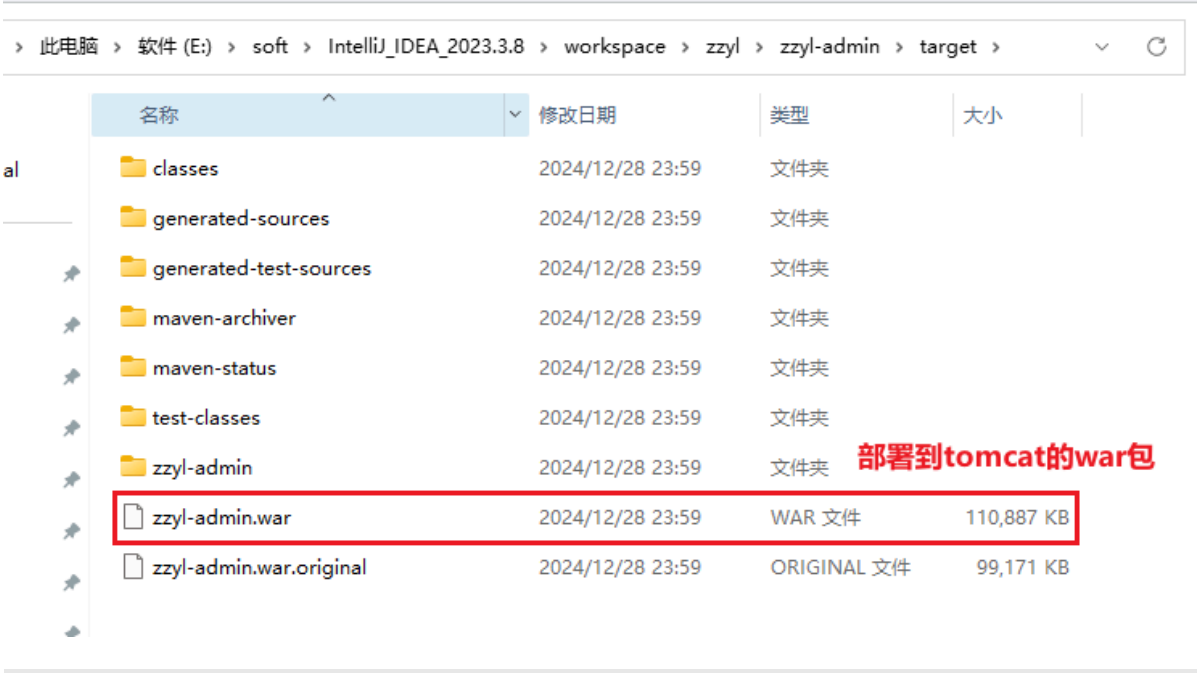

```
[INFO] --- spring-boot-maven-plugin:2.5.15:repackage (default) @ zzyl-admin ---
[INFO] Replacing main artifact with repackaged archive
[INFO] ------------------------------------------------------------------------
[INFO] Reactor Summary for zzyl 3.8.8:
[INFO]
[INFO] zzyl ........................................... SUCCESS [  0.002 s]
[INFO] zzyl-common .................................... SUCCESS [  6.157 s]
[INFO] zzyl-system .................................... SUCCESS [  1.250 s]
[INFO] zzyl-framework ................................. SUCCESS [  1.521 s]
[INFO] zzyl-quartz .................................... SUCCESS [  0.816 s]
[INFO] zzyl-generator ................................. SUCCESS [  1.360 s]
[INFO] zzyl-oss ....................................... SUCCESS [  0.624 s]
[INFO] zzyl-nursing-platform .......................... SUCCESS [  2.152 s]
[INFO] zzyl-admin ..................................... SUCCESS [  2.818 s]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  17.080 s
[INFO] Finished at: 2024-12-28T23:20:02+08:00
[INFO] ------------------------------------------------------------------------
```

查看核心工程: zzyl-admin的包


zzyl E:\soft\IntelliJ_IDEA_2023.3.8\workspace\zzyl
.github
.idea
bin
doc
sql
zzyl-admin
src
target
classes
generated-sources
generated-test-sources
maven-archiver
maven-status
test-classes
zzyl-admin
zzyl-admin.war
可以看到war包
zzyl-admin.war.original
deploy.sh
Dockerfile
pom.xml

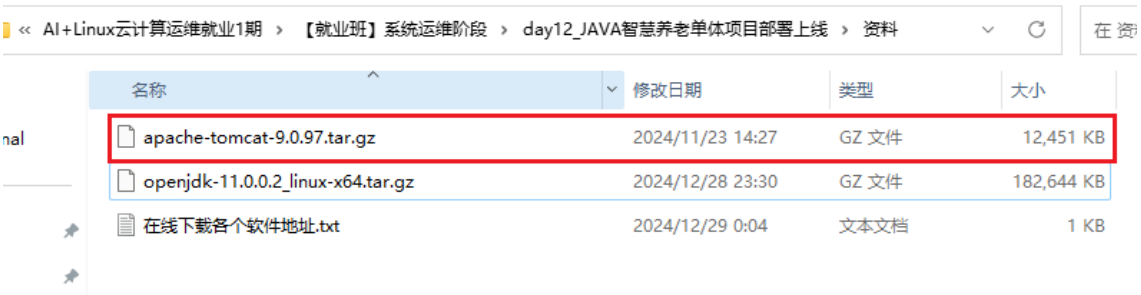
打开该jar包路径：


Project
zzyl E:\soft\IntelliJ_IDEA_2023.3.8\workspace\zzyl
.github
.idea
bin
doc
sql
zzyl-admin
src
target
classes
generated-sources
generated-test-sources
maven-archiver
maven-status
test-classes
zzyl-admin
右键
zzyl-admin.wa
zzyl-admin.wa
deploy.sh
Dockerfile
pom.xml
zzyl-common
zzyl-framework
zzyl-generator
pom.xml (zzyl-admin)
11 <packaging>
12 <artifactId
13
14 <descriptio
15 web服务
16 </descripti
17
18 <dependenci
19
20 <!-- sp
21 <depend
22 <gr
<ar
<op
</depen
description
Run
zzyl [package]
zzyl [package]: At 202
] zzyl-system .....
] zzyl-framework ...
] zzyl-quartz
New
Cut Ctrl+X
Copy Ctrl+C
Copy Path/Reference...
Paste Ctrl+V
Find Usages Alt+F7
Analyze
Refactor
Bookmarks
Delete... Delete
Override File Type
Open in Right Split Shift+Enter
Open In
Local History
Repair IDE on File
Explorer
Path Popup Ctrl+Alt+F12
Terminal



尝试运行：

- 1- 安装Tomcat服务器：



  - 1.1 上传Tomcat到node2的 /export/software下 或 联网下载

```
wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.98/bin/apache-tomcat-9.0.98.tar.gz
```

```
[root@node2 software]# pwd
/export/software
[root@node2 software]# ll
总用量 12452
-rw-r--r-- 1 root root 12749071 12月 29 00:12 apache-tomcat-9.0.97.tar.gz
```

  - 1.2- 解压Tomcat服务器到/usr/local目录下

```
cd /export/software/
tar -xzf apache-tomcat-9.0.97.tar.gz -C /usr/local/

cd /usr/local/
```

```
[root@node1 software]# cd /usr/local/
[root@node1 local]#
[root@node1 local]# ll
total 8
drwxr-xr-x. 9 root  root  4096 Nov 23 14:31 apache-tomcat-9.0.97
drwxr-xr-x. 2 root  root     6 Jun 25 22:23 bin
drwxr-xr-x. 2 root  root     6 Jun 25 22:23 etc
drwxr-xr-x. 2 root  root     6 Jun 25 22:23 games
drwxr-xr-x. 2 root  root     6 Jun 25 22:23 include
drwxr-xr-x. 7 10143 10143 4096 Dec 11  2019 jdk1.8.0_241
drwxr-xr-x. 2 root  root     6 Jun 25 22:23 lib
drwxr-xr-x. 3 root  root    17 Oct 12 16:27 lib64
drwxr-xr-x. 2 root  root     6 Jun 25 22:23 libexec
drwxr-xr-x. 2 root  root     6 Jun 25 22:23 sbin
drwxr-xr-x. 5 root  root    49 Oct 12 16:27 share
drwxr-xr-x. 2 root  root     6 Jun 25 22:23 src
```

tomcat目录介绍：

```
[root@node1 apache-tomcat-9.0.97]# ll
total 136
drwxr-x---. 2 root root  4096 Nov 23 14:31 bin
-rw-r-----. 1 root root 20913 Nov  7 03:55 BUILDING.txt
drwx------. 2 root root  4096 Nov  7 03:55 conf
-rw-r-----. 1 root root  6166 Nov  7 03:55 CONTRIBUTING.md
drwxr-x---. 2 root root  4096 Nov 23 14:31 lib
-rw-r-----. 1 root root 57092 Nov  7 03:55 LICENSE
drwxr-x---. 2 root root     6 Nov  7 03:55 logs
-rw-r-----. 1 root root  2333 Nov  7 03:55 NOTICE
-rw-r-----. 1 root root  3283 Nov  7 03:55 README.md
-rw-r-----. 1 root root  6901 Nov  7 03:55 RELEASE-NOTES
-rw-r-----. 1 root root 16538 Nov  7 03:55 RUNNING.txt
drwxr-x---. 2 root root    30 Nov 23 14:31 temp
drwxr-x---. 7 root root    81 Nov  7 03:55 webapps
drwxr-x---. 2 root root     6 Nov  7 03:55 work
```

执行脚本： 启动、关闭等

Tomcat服务器配置文件

相关依赖包

日志信息

web应用部署目录

- 1.3上传中州养老后台管理系统项目到webapps目录下
  - 6.1 上传至webapps目录下

```
[root@node2 webapps]# pwd
/usr/local/apache-tomcat-9.0.97/webapps
[root@node2 webapps]# ll
总用量 110896
drwxr-x--- 16 root root      4096 12月 29 00:18 docs
drwxr-x---  7 root root        99 12月 29 00:18 examples
drwxr-x---  6 root root        79 12月 29 00:18 host-manager
drwxr-x---  6 root root       114 12月 29 00:18 manager
drwxr-x---  3 root root      4096 12月 29 00:18 ROOT
-rw-r--r--  1 root root 113548121 12月 29 00:18 zzyl-admin.war
[root@node2 webapps]#
```

  - 6.2 启动tomcat服务器

```
cd /usr/local/apache-tomcat-9.0.97
./bin/shutdown.sh
./bin/startup.sh
```

```
[root@node2 apache-tomcat-9.0.97]# ./bin/startup.sh
Using CATALINA_BASE:   /usr/local/apache-tomcat-9.0.97
Using CATALINA_HOME:   /usr/local/apache-tomcat-9.0.97
Using CATALINA_TMPDIR: /usr/local/apache-tomcat-9.0.97/temp
Using JRE_HOME:        /root/jdk-11.0.0.2
Using CLASSPATH:       /usr/local/apache-tomcat-9.0.97/bin/bootstrap.jar:/usr/local/apache-tomcat-9.0.97/bin/tomcat-juli.ja
r
Using CATALINA_OPTS:
```

查看日志： tail -100f logs/catalina.out

即可看到：



可以通过日志， 观察启动过程中是否有报错即可

此时我们再看webapps目录下

```
[root@node2 apache-tomcat-9.0.97]# cd webapps/
[root@node2 webapps]# ll
总用量 110896
drwxr-x--- 16 root root      4096 12月 29 00:18 docs
drwxr-x---  7 root root        99 12月 29 00:18 examples
drwxr-x---  6 root root        79 12月 29 00:18 host-manager
drwxr-x---  6 root root       114 12月 29 00:18 manager
drwxr-x---  3 root root      4096 12月 29 00:18 ROOT
drwxr-x---  5 root root        48 12月 29 00:20 zzyl-admin
-rw-r--r--  1 root root 113548121 12月 29 00:18 zzyl-admin.war
[root@node2 webapps]#
```

说明：这时候就有了一个zzyl-admin的目录。 注意： 后续访问该项目 需要增加zzyl-admin的名称 才可以

例如：http://node2的ip地址:8080/zzyl-admin 这是项目的标准路径

如果希望为：http://node2的ip地址:8080/

可以将项目的名称更改为ROOT即可，类似于我们基础班，但是由于该项目仅为后台服务， 并无前端页面，对用户访问不会产生影响， 故本次不设置也OK

8. 前端系统部署操作

8.1 在Linux中安装Nginx服务器

如何部署Nginx服务器呢？ 【node3】

- 1- 安装Nginx服务器

```
dnf install nginx -y
```

- 2- 启动并启用 Nginx 服务

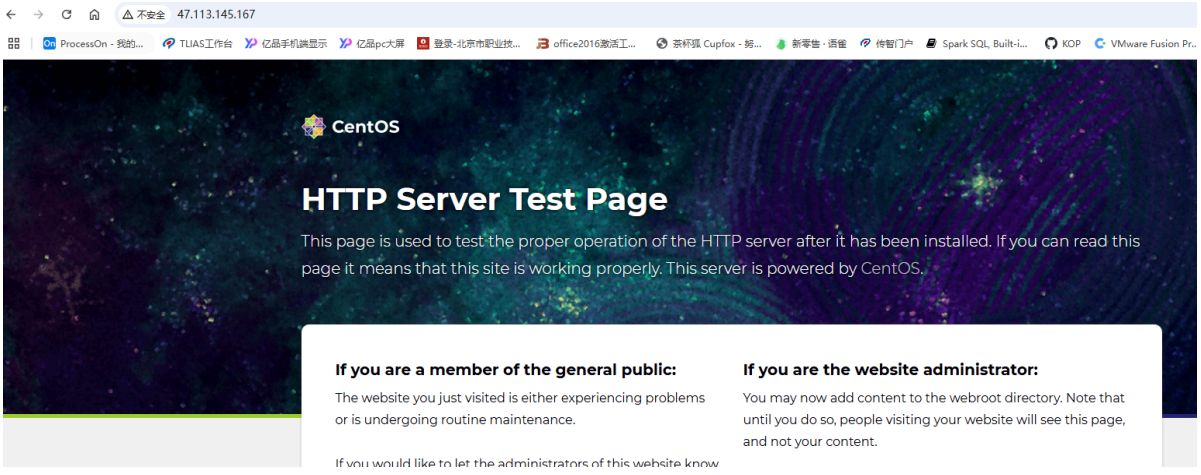
```
systemctl start nginx
systemctl enable nginx
# 查看状态信息
systemctl status nginx
```

```
[root@node3 ~]# systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
     Active: active (running) since Sun 2024-12-29 01:27:47 CST; 3s ago
   Main PID: 3882 (nginx)
      Tasks: 5 (limit: 22929)
     Memory: 4.9M
        CPU: 27ms
     CGroup: /system.slice/nginx.service
             ├─3882 "nginx: master process /usr/sbin/nginx"
             ├─3883 "nginx: worker process"
             ├─3884 "nginx: worker process"
             ├─3885 "nginx: worker process"
             └─3886 "nginx: worker process"

12月 29 01:27:47 node3.itcast.cn systemd[1]: Starting The nginx HTTP and reverse proxy server...
12月 29 01:27:47 node3.itcast.cn nginx[3880]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
12月 29 01:27:47 node3.itcast.cn nginx[3880]: nginx: configuration file /etc/nginx/nginx.conf test is successful
12月 29 01:27:47 node3.itcast.cn systemd[1]: Started The nginx HTTP and reverse proxy server.
```

- 3- 测试是否可以访问

```
http://node3公网IP
```

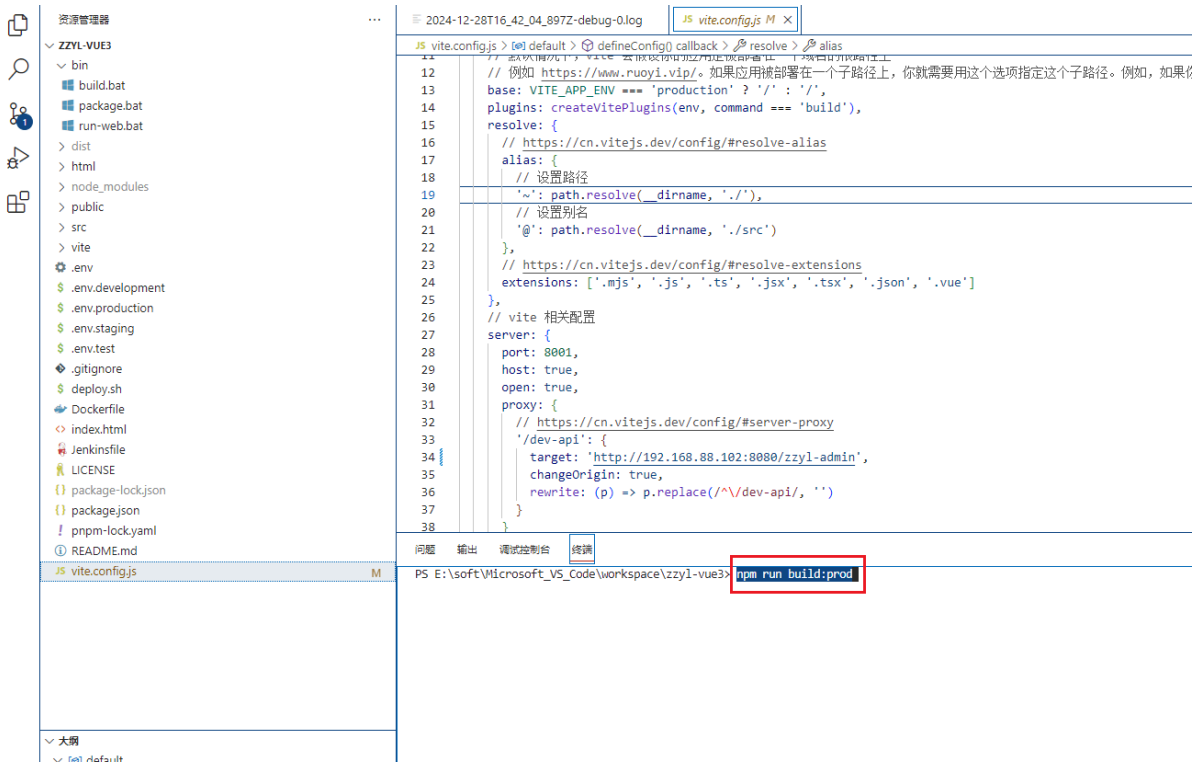


能看到Nginx 的欢迎页面，说明 Nginx 已成功部署

8.2 对前端项目进行打包

- 1- 在终端中， 执行打包命令

```
npm run build:prod
```



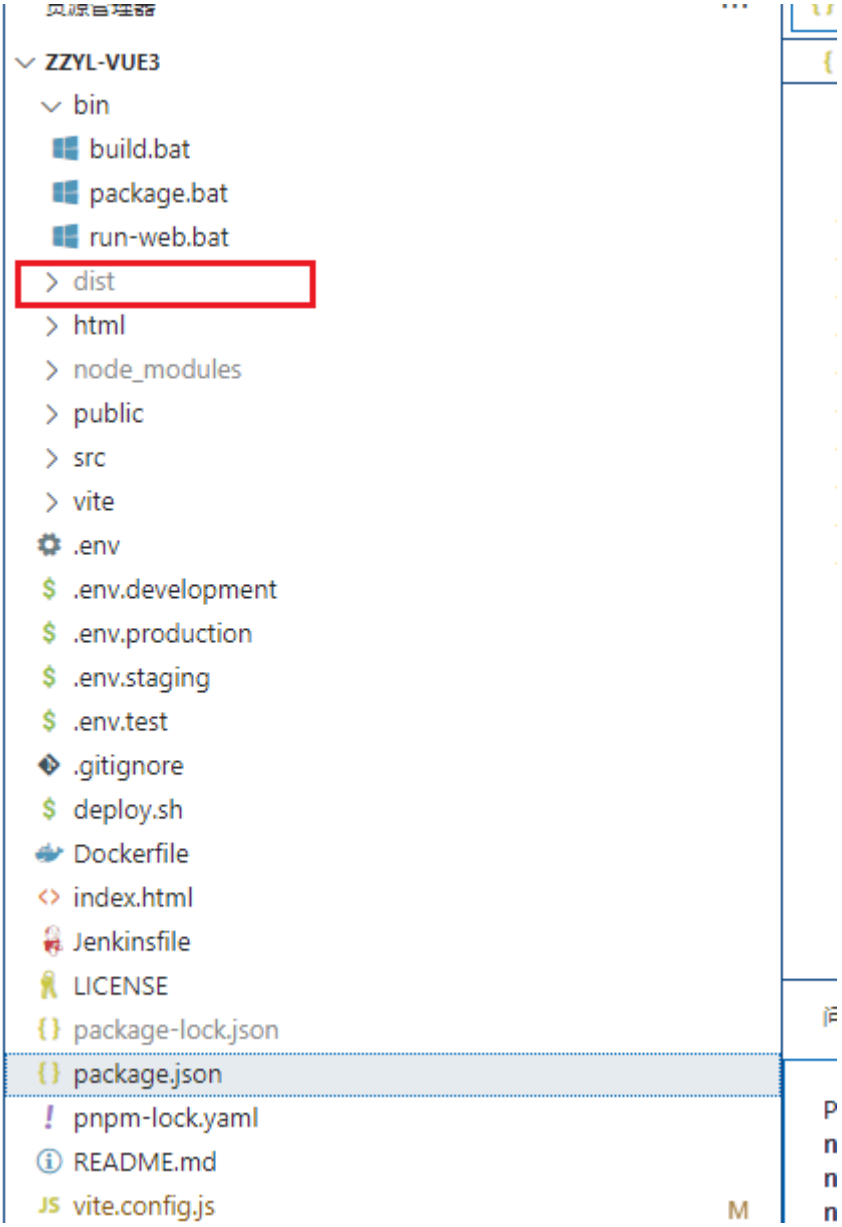
使用当前命令的原因： 开发人员在package.json 这个核心配置文件中， 定义了如何打包， 使用什么命令



```
package.json 中的 scripts 配置， 主要是为了简化和标准化常用的命令操作，项目已经定义了多个构建命令，分别为：

dev: 启动开发服务器，使用 Vite（运行 vite 命令）。
build:prod: 打包项目用于生产环境（运行 vite build）。
build:stage: 打包项目用于 staging 环境（运行 vite build --mode staging） 构建预发布（阶段环境）代码
preview: 预览生产构建后的内容（运行 vite preview）。
```

打包执行完成后， 在项目中就会出来一个dist目录，该文件夹内将包含 HTML、CSS、JavaScript 等资源文件，准备部署到生产环境。





8.3 在Nginx中配置操作

- 1- 将dist目录上传到/var/www/目录下，如果目标路径不存在， 手动创建

```
mkdir -p /var/www/

将disk上传到此目录下
```

```
[root@node3 www]# pwd
/var/www
[root@node3 www]# ll
总用量 0
drwxr-xr-x 3 root root 101 12月 29 01:48 dist
[root@node3 www]#
```

- 2- 配置Nginx的配置文件

```
vim /etc/nginx/nginx.conf

# 添加相关内容：
# 在HTTP的目录下， 添加以下三个add_header   CORS 配置
add_header Access-Control-Allow-Origin *;
add_header Access-Control-Allow-Methods "GET, POST, OPTIONS, PUT, DELETE";
add_header Access-Control-Allow-Headers "Origin, X-Requested-With, Content-Type,
Accept, Authorization";


作用：
    这几行配置用于设置 跨域资源共享（CORS），它们在 Nginx 中用来允许其他域名访问你的资源。CORS 是一种机制，允许通过浏览器发起跨域 HTTP 请求，常用于 Web 应用与不同域名的 API 交互时，解决浏览器的同源策略限制。
```

```
http {
    log_format  main  '$remote_addr - $remote_user [$time_local] "$request" '
                      '$status $body_bytes_sent "$http_referer" '
                      '"$http_user_agent" "$http_x_forwarded_for"';

    access_log  /var/log/nginx/access.log  main;

    sendfile            on;
    tcp_nopush          on;
    tcp_nodelay         on;
    keepalive_timeout   65;
    types_hash_max_size 4096;

    include             /etc/nginx/mime.types;
    default_type        application/octet-stream;

    # Load modular configuration files from the /etc/nginx/conf.d directory.
    # See http://nginx.org/en/docs/ngx_core_module.html#include
    # for more information.
    include /etc/nginx/conf.d/*.conf;                                    关注放置位置

    add_header Access-Control-Allow-Origin *;
    add_header Access-Control-Allow-Methods "GET, POST, OPTIONS, PUT, DELETE";
    add_header Access-Control-Allow-Headers "Origin, X-Requested-With, Content-Type, Accept, Authorization";


    server {
        listen       80;
```

在server中， 添加以下三行内容， 删除原有的server_name 和root

```
client_max_body_size 60m; # 限制客户端请求体的最大大小。
client_body_buffer_size 512k; # 设置 Nginx 用于缓冲客户端请求体的内存大小。
client_header_buffer_size 2k; # 设置 Nginx 用于缓冲请求头的内存大小。
```

说明

这些配置项与 Nginx 处理客户端请求的请求体大小、请求头缓冲区等相关，主要用于控制 Nginx 在处理上传文件、请求体内容和请求头时的行为

```
    server {
        listen       80;
        listen       [::]:80;

        client_max_body_size 60m;
        client_body_buffer_size 512k;
        client_header_buffer_size 2k;
        # Load configuration files for the default server block.
        include /etc/nginx/default.d/*.conf;

        error_page 404 /404.html;
        location = /404.html {
        }

        error_page 500 502 503 504 /50x.html;
        location = /50x.html {
        }
    }
```

在server中，继续添加以下内容， 注意根据图片 确定放置位置

```
        location / {
                root   /var/www/dist;
                index  index.html index.htm;
                proxy_set_header   Upgrade          $http_upgrade;
                proxy_set_header   Connection       upgrade;
                try_files $uri $uri/ /index.html;
         }

         location /prod-api/ {
                proxy_pass http://tomcat对应的阿里云服务器私网IP地址:8080/zzyl-
admin/;
```

```
                proxy_set_header    Upgrade           $http_upgrade;
                proxy_set_header    Connection        upgrade;
        }
```

```
        listen       80;
        listen       [::]:80;

        client_max_body_size 60m;
        client_body_buffer_size 512k;
        client_header_buffer_size 2k;

        # Load configuration files for the default server block.
        include /etc/nginx/default.d/*.conf;

        error_page 404 /404.html;
        location = /404.html {
        }

        error_page 500 502 503 504 /50x.html;
        location = /50x.html {
        }

        location / {
                root   /var/www/dist;
                index  index.html index.htm;
                proxy_set_header    Upgrade           $http_upgrade;
                proxy_set_header    Connection        upgrade;
                try_files $uri $uri/ /index.html;
        }

        location /prod-api/ {
                proxy_pass http://172.29.0.30:8080/zzyl-admin/;
                proxy_set_header    Upgrade           $http_upgrade;
                proxy_set_header    Connection        upgrade;
        }

    }

# Settings for a TLS enabled server.
```

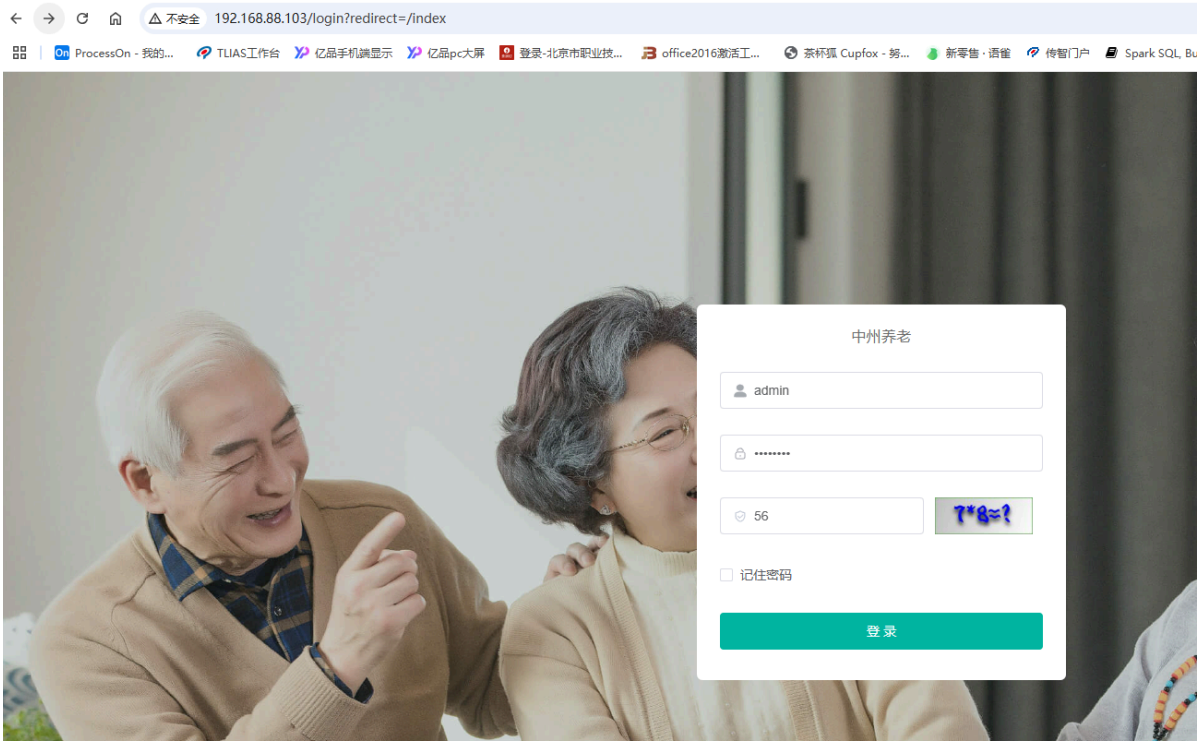
- 3- 重启Nginx服务

```
重启 Nginx:
systemctl restart nginx

或
重新加载 Nginx 配置（不停止服务）:
nginx -s reload
```

```
[root@node3 www]# nginx -s reload
[root@node3 www]#
```

- 4- 刷新浏览器： 访问192.168.88.103



9- 日志服务配置

- 0 - noed1挂载一块数据盘: 可采用此方案



- 1- 安装数据同步服务【三个节点均需要安装】

```
dnf install rsync -y
```

- 2- 配置定时操作

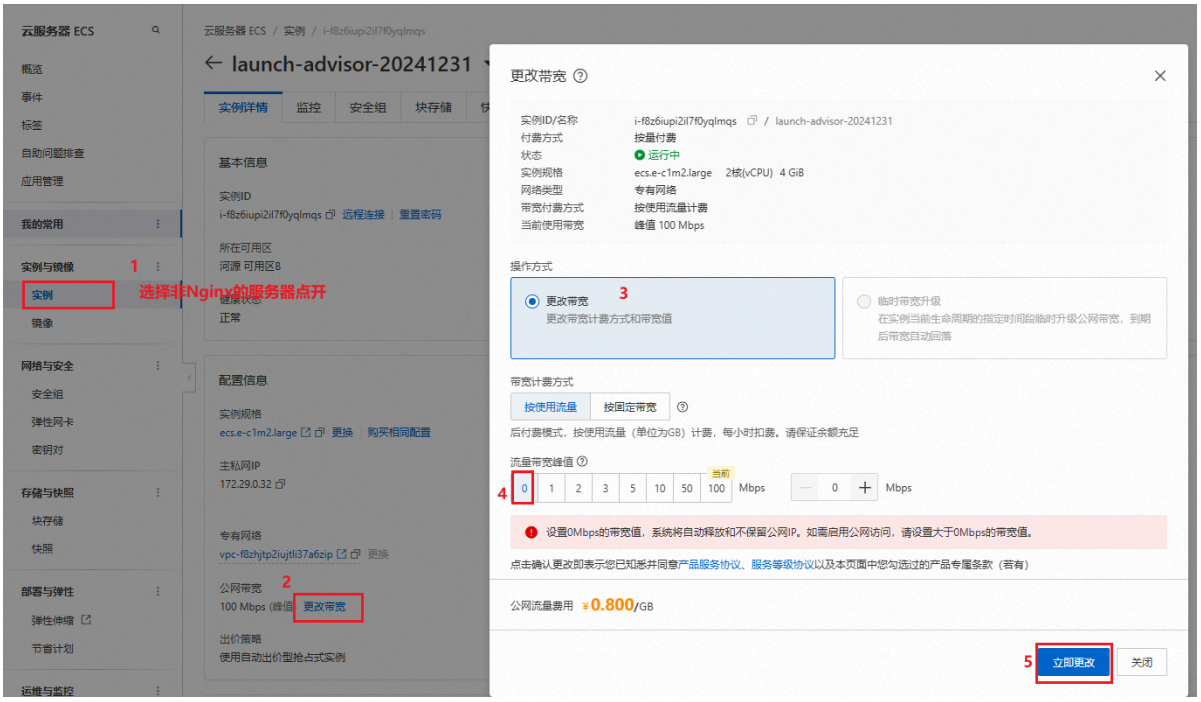
```
mkdir -p /mnt/zzyl_logs/ 【临时创建， 正常挂载后， 就会有对应的目录 即可直接使用】
mkdir -p /mnt/zzyl_logs/tomcat
mkdir -p /mnt/zzyl_logs/nginx


crontab -e

配置:
0 1 * * * /usr/bin/rsync -avz root@172.29.0.30:/home/ruoyi/logs/sys-*.*.log
/mnt/zzyl_logs/tomcat
0 1 * * * /usr/bin/rsync -avz root@172.29.0.31:/var/log/nginx/*.log-*
/mnt/zzyl_logs/nginx
```

10- 删除阿里云公网IP访问

说明: 由于 仅使用node3来完成外部访问工作, node2中由于有阿里云OSS和百度千帆大模型需要连接外部网络, 也不建议去除, 顾需要去除node1的公网IP 防止外部网络访问,提高安全性



11- 阿里云安全组设置





其他端口均不开放

12- 完成后的操作

整体上线完成后 就可以正式释放了, 注意: 各个服务器释放后, 如果涉及到添加云盘的情况, 需要将云盘也要释放掉